



Get Started with the CDM for Collateral

Module D – How the CDM is used

Version 2.0 – Feb 2025

Click below to get started!

Start

There are six modules in this Get Started guide:

A	Introduction to the CDM	15 minutes	Complete
B	Accessing the CDM	10 minutes	Complete
C	How the CDM works	30 minutes	Complete
D	How the CDM is used	45 minutes	Start
E	Collateral Use Cases	45 minutes	Start
F	Advanced Features	30 minutes	Start

D

The fourth module in this series explains how the CDM is used within financial services to deliver the stated benefits.

1

Overview of the CDM and the CDM components

Start

2

Data Interfaces to the CDM and JSON

Start

3

Documentation and The Object Builder

Start

4

Introduction to Functions

Start

5

The CDM for Technologists

Start

- In the first module of this Get Started guide, we covered some of the benefits of the CDM, including:
 - enables interoperability between systems/services
 - removes burden of setting up connections to different systems/entities, laying groundwork for STP
 - promotes transparency and alignment between regulators and market participants.
- And, we also described the CDM as a “standardized, machine-readable, and machine-executable model”.

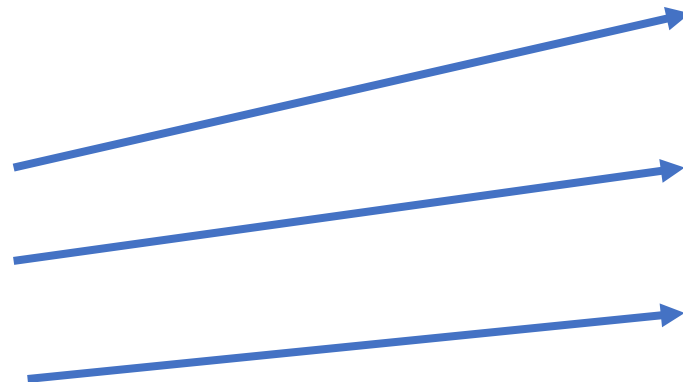
This module focuses on these benefits and explains:

- Standardized
- Machine-readable
- Machine-executable.

- enables interoperability between systems/services
- removing burden of setting up connections to different systems/entities, laying groundwork for STP
- promotes transparency and alignment between regulators and market participants.

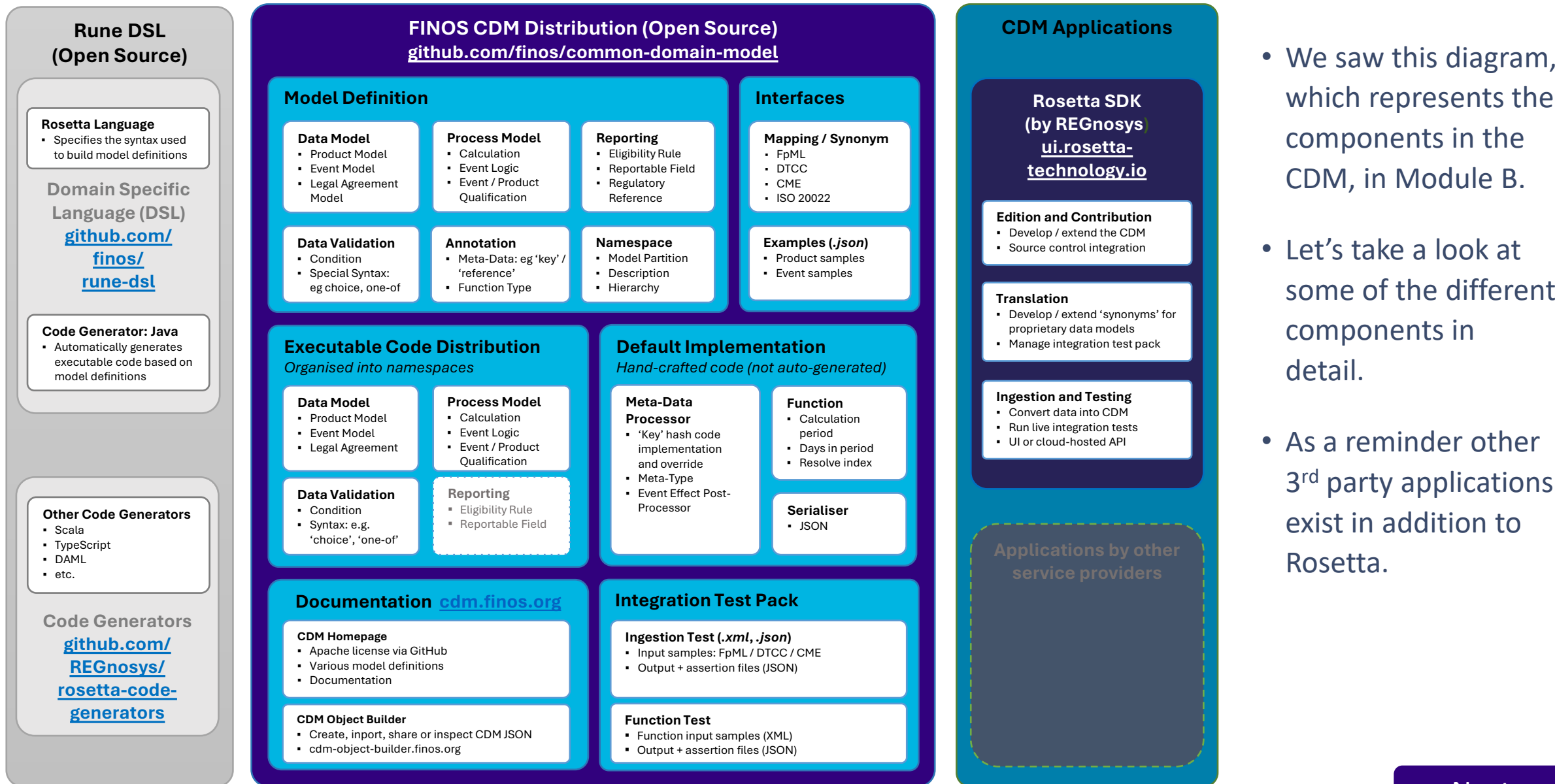


- Standardized
- Machine-readable
- Machine-executable.



What the CDM delivers to provide this:

- Definitions of data definitions that can be adopted between systems and services.
- A focus on the semantics (ie the meaning of common products, processes and events) rather than physical formats bound to a specific technology.
- Freely available with an open-source licence but backed by market players and 3 Trade Associations.
- A common, available definition for the model with a governance processes for change and updates.
- The ability to express CDM data in a structured format that can be read by a computer.
- The model includes the representation of business events and processes, not just data formats.



- We saw this diagram, which represents the components in the CDM, in Module B.
- Let's take a look at some of the different components in detail.
- As a reminder other 3rd party applications exist in addition to Rosetta.

FINOS CDM Distribution (Open Source)
github.com/finos/common-domain-model

Model Definition

Data Model

- Product Model
- Event Model
- Legal Agreement Model

Process Model

- Calculation
- Event Logic
- Event / Product Qualification

Reporting

- Eligibility Rule
- Reportable Field
- Regulatory Reference

Data Validation

- Condition
- Special Syntax: eg choice, one-of

Annotation

- Meta-Data: eg 'key' / 'reference'
- Function Type

Namespace

- Model Partition
- Description
- Hierarchy

- As we have seen, the CDM is distributed as Open Source by FINOS via GitHub. Let's look at the key component, the Model Definition:
- Within the **Data** model, we have seen how products can be represented using *data types* which can include data validation conditions.
- We will review the **Process** model (handling calculation, product qualification and events) in a few moments.
- **Reporting** definitions are used extensively in ISDA's Digital Regulatory Reporting capability, which uses these capabilities to enable efficient implementation of trade reporting rules and rule amendments of multiple global jurisdictions.
- We will also take a brief look at **Annotations** and **Functions** in this module.

FINOS CDM Distribution (Open Source)

github.com/finos/common-domain-model

Model Definition

Data Model

- Product Model
- Event Model
- Legal Agreement Model

Process Model

- Calculation
- Event Logic
- Event / Product Qualification

Reporting

- Eligibility Rule
- Reportable Field
- Regulatory Reference

Data Validation

- Condition
- Special Syntax: eg choice, one-of

Annotation

- Meta-Data: eg 'key' / 'reference'
- Function Type

Namespace

- Model Partition
- Description
- Hierarchy

Interfaces

Mapping / Synonym

- FpML
- DTCC
- CME
- ISO 20022

Examples (.json)

- Product samples
- Event samples

Integration Test Pack

Ingestion Test (.xml, .json)

- Input samples: FpML / DTCC / CME
- Output + assertion files (JSON)

Function Test

- Function input samples (XML)
- Output + assertion files (JSON)

- The Model Definition is a set of logical descriptions of Products, Events and Process, but is brought to life in an implementation with real data.
- The CDM includes **Interface** components for both inbound and outbound data, which is usually represented in **JSON**, which we will look at shortly.
- The CDM is also bundled with a set of **Integration Tests** including test data (for Products and Events, input and outputs, and Functions) and this will be covered briefly in an overview of some of the features available to the more technically inclined.

FINOS CDM Distribution (Open Source)

github.com/finos/common-domain-model

Model Definition

Data Model

- Product Model
- Event Model
- Legal Agreement Model

Process Model

- Calculation
- Event Logic
- Event / Product Qualification

Reporting

- Eligibility Rule
- Reportable Field
- Regulatory Reference

Data Validation

- Condition
- Special Syntax: eg choice, one-of

Annotation

- Meta-Data: eg 'key' / 'reference'
- Function Type

Namespace

- Model Partition
- Description
- Hierarchy

Interfaces

Mapping / Synonym

- FpML
- DTCC
- CME
- ISO 20022

Examples (.json)

- Product samples
- Event samples

Documentation cdm.finos.org

CDM Homepage

- Apache license via GitHub
- Various model definitions
- Documentation

CDM Object Builder

- Create, import, share or inspect CDM JSON
- cdm-object-builder.finos.org

Integration Test Pack

Ingestion Test (.xml, .json)

- Input samples: FpML / DTCC / CME
- Output + assertion files (JSON)

Function Test

- Function input samples (XML)
- Output + assertion files (JSON)

- There is also a comprehensive set of **Documentation** on the model and how it works.
- And the **CDM Object Builder** is a tool designed to aid understanding and adoption of the model.

FINOS CDM Distribution (Open Source) github.com/finos/common-domain-model

Model Definition

Data Model

- Product Model
- Event Model
- Legal Agreement Model

Process Model

- Calculation
- Event Logic
- Event / Product Qualification

Reporting

- Eligibility Rule
- Reportable Field
- Regulatory Reference

Data Validation

- Condition
- Special Syntax: eg choice, one-of

Annotation

- Meta-Data: eg 'key' / 'reference'
- Function Type

Namespace

- Model Partition
- Description
- Hierarchy

Interfaces

Mapping / Synonym

- FpML
- DTCC
- CME
- ISO 20022

Examples (.json)

- Product samples
- Event samples

Executable Code Distribution

Organised into namespaces

Data Model

- Product Model
- Event Model
- Legal Agreement

Process Model

- Calculation
- Event Logic
- Event / Product Qualification

Data Validation

- Condition
- Syntax: e.g. 'choice', 'one-of'

Reporting

- Eligibility Rule
- Reportable Field

Default Implementation

Hand-crafted code (not auto-generated)

Meta-Data Processor

- 'Key' hash code implementation and override
- Meta-Type
- Event Effect Post-Processor

Function

- Calculation period
- Days in period
- Resolve index

Serialiser

- JSON

Documentation cdm.finos.org

CDM Homepage

- Apache license via GitHub
- Various model definitions
- Documentation

CDM Object Builder

- Create, inport, share or inspect CDM JSON
- cdm-object-builder.finos.org

Integration Test Pack

Ingestion Test (.xml, .json)

- Input samples: FpML / DTCC / CME
- Output + assertion files (JSON)

Function Test

- Function input samples (XML)
- Output + assertion files (JSON)

- As we have already described, the CDM is more than a data model, it is also a Process model.
- To enable wide-spread and convenient adoption, the FINOS CDM Distribution includes executable code that substantiates the Data Model, Process Model and Event Model.
- The **Executable Code Distribution** includes automatically generated code for the core CDM in a variety of software programming languages.
- The **Default Implementation** includes additional supporting code to substantiate the model written in Java.

Rune DSL (Open Source)

Rune Language

- Specifies the syntax used to build model definitions

Domain Specific Language (DSL)

github.com/finos/rune-dsl

Code Generator: Java

- Automatically generates executable code based on model definitions

Other Code Generators

- Scala
- TypeScript
- DAML
- etc.

Code Generators

github.com/REGnosys/rosetta-code-generators

FINOS CDM Distribution (Open Source)

github.com/finos/common-domain-model

Model Definition

Data Model

- Product Model
- Event Model
- Legal Agreement Model

Process Model

- Calculation
- Event Logic
- Event / Product Qualification

Reporting

- Eligibility Rule
- Reportable Field
- Regulatory Reference

Data Validation

- Condition
- Special Syntax: eg choice, one-of

Annotation

- Meta-Data: eg 'key' / 'reference'
- Function Type

Namespace

- Model Partition
- Description
- Hierarchy

Interfaces

Mapping / Synonym

- FpML
- DTCC
- CME
- ISO 20022

Examples (.json)

- Product samples
- Event samples

Executable Code Distribution

Organised into namespaces

Data Model

- Product Model
- Event Model
- Legal Agreement

Process Model

- Calculation
- Event Logic
- Event / Product Qualification

Data Validation

- Condition
- Syntax: e.g. 'choice', 'one-of'

Reporting

- Eligibility Rule
- Reportable Field

Default Implementation

Hand-crafted code (not auto-generated)

Meta-Data Processor

- 'Key' hash code implementation and override
- Meta-Type
- Event Effect Post-Processor

Function

- Calculation period
- Days in period
- Resolve index

Serialiser

- JSON

Documentation cdm.finos.org

CDM Homepage

- Apache license via GitHub
- Various model definitions
- Documentation

CDM Object Builder

- Create, inport, share or inspect CDM JSON
- cdm-object-builder.finos.org

Integration Test Pack

Ingestion Test (.xml, .json)

- Input samples: FpML / DTCC / CME
- Output + assertion files (JSON)

Function Test

- Function input samples (XML)
- Output + assertion files (JSON)

- The **Rune Domain Specific Language** is the specification of the syntax used to build the model definitions.
- The DSL is code too, written in Java.
- The DSL is owned by the technology firm REGnosys.
- It is also open source, available in GitHub, although it is not part of the FINOS distribution.

Rune DSL (Open Source)

Rosetta Language

- Specifies the syntax used to build model definitions

Domain Specific Language (DSL)

github.com/finos/rune-dsl

Code Generator: Java

- Automatically generates executable code based on model definitions

Other Code Generators

- Scala
- TypeScript
- DAML
- etc.

Code Generators

github.com/REGnosys/rosetta-code-generators

FINOS CDM Distribution (Open Source)

github.com/finos/common-domain-model

Model Definition

Data Model

- Product Model
- Event Model
- Legal Agreement Model

Process Model

- Calculation
- Event Logic
- Event / Product Qualification

Reporting

- Eligibility Rule
- Reportable Field
- Regulatory Reference

Data Validation

- Condition
- Special Syntax: eg choice, one-of

Annotation

- Meta-Data: eg 'key' / 'reference'
- Function Type

Namespace

- Model Partition
- Description
- Hierarchy

Interfaces

Mapping / Synonym

- FpML
- DTCC
- CME
- ISO 20022

Examples (.json)

- Product samples
- Event samples

Executable Code Distribution

Organised into namespaces

Data Model

- Product Model
- Event Model
- Legal Agreement

Process Model

- Calculation
- Event Logic
- Event / Product Qualification

Data Validation

- Condition
- Syntax: e.g. 'choice', 'one-of'

Reporting

- Eligibility Rule
- Reportable Field

Default Implementation

Hand-crafted code (not auto-generated)

Meta-Data Processor

- 'Key' hash code implementation and override
- Meta-Type
- Event Effect Post-Processor

Function

- Calculation period
- Days in period
- Resolve index

Serialiser

- JSON

Documentation cdm.finos.org

CDM Homepage

- Apache license via GitHub
- Various model definitions
- Documentation

CDM Object Builder

- Create, import, share or inspect CDM JSON
- cdm-object-builder.finos.org

Integration Test Pack

Ingestion Test (.xml, .json)

- Input samples: FpML / DTCC / CME
- Output + assertion files (JSON)

Function Test

- Function input samples (XML)
- Output + assertion files (JSON)

CDM Applications

Rosetta SDK (by REGnosys)

ui.rosetta-technology.io

Edition and Contribution

- Develop / extend the CDM
- Source control integration

Translation

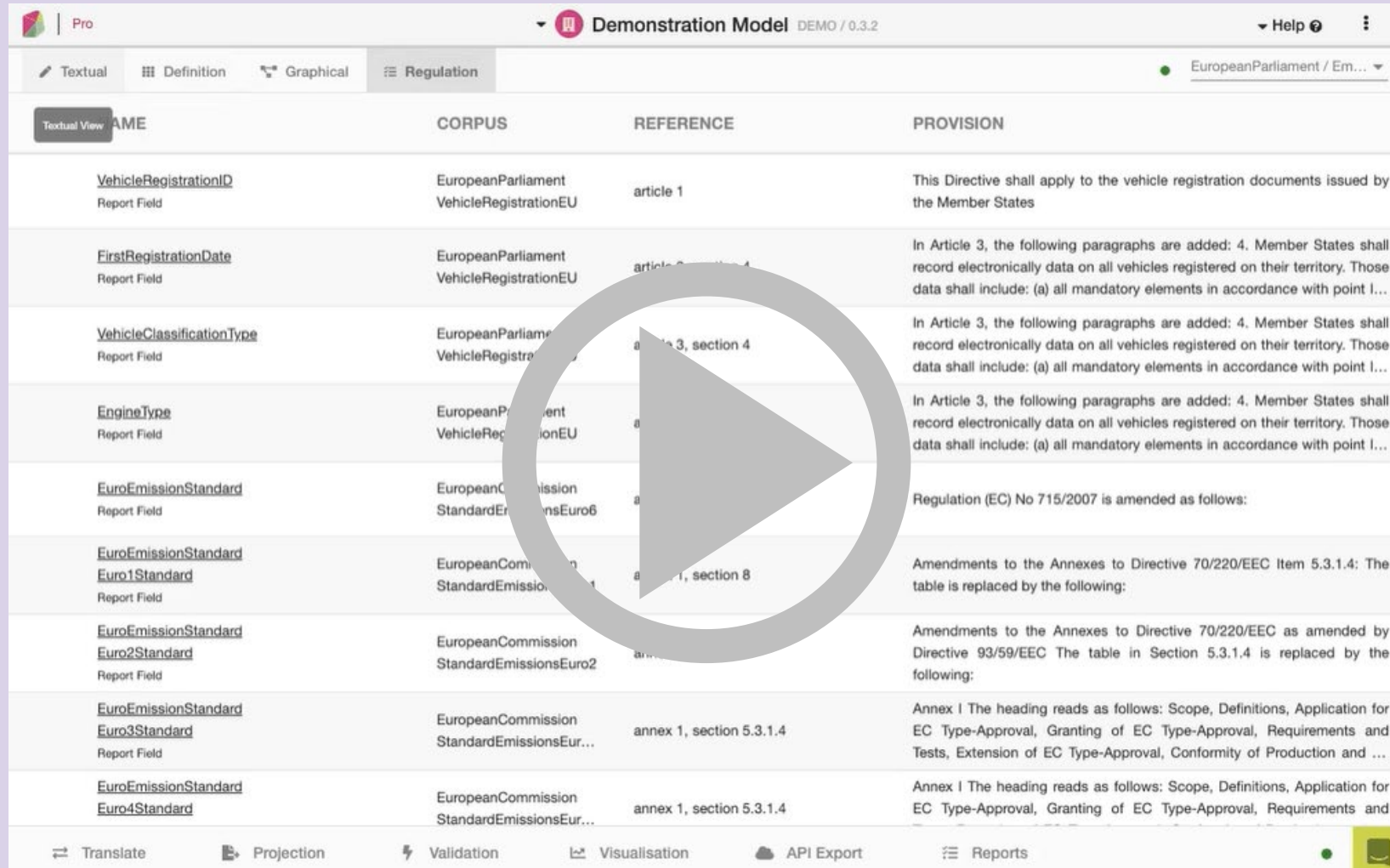
- Develop / extend 'synonyms' for proprietary data models
- Manage integration test pack

Ingestion and Testing

- Convert data into CDM
- Run live integration tests
- UI or cloud-hosted API

Applications by other service providers

- As a set of open source components, members and fintech vendors can create their own capabilities, implementations, and applications.
- REGnosys provides the community with free access to the **Rosetta SDK Platform** to view the CDM.
- ISDA does not endorse any particular vendor.



The screenshot shows the Rosetta Demonstration Model interface. It features a top navigation bar with 'Pro', 'Demonstration Model DEMO / 0.3.2', and 'Help'. Below this is a tabbed interface with 'Textual', 'Definition', 'Graphical', and 'Regulation' tabs. The 'Regulation' tab is active, showing a table with columns: NAME, CORPUS, REFERENCE, and PROVISION. A large play button is overlaid on the table. The table lists various report fields and their corresponding legal references.

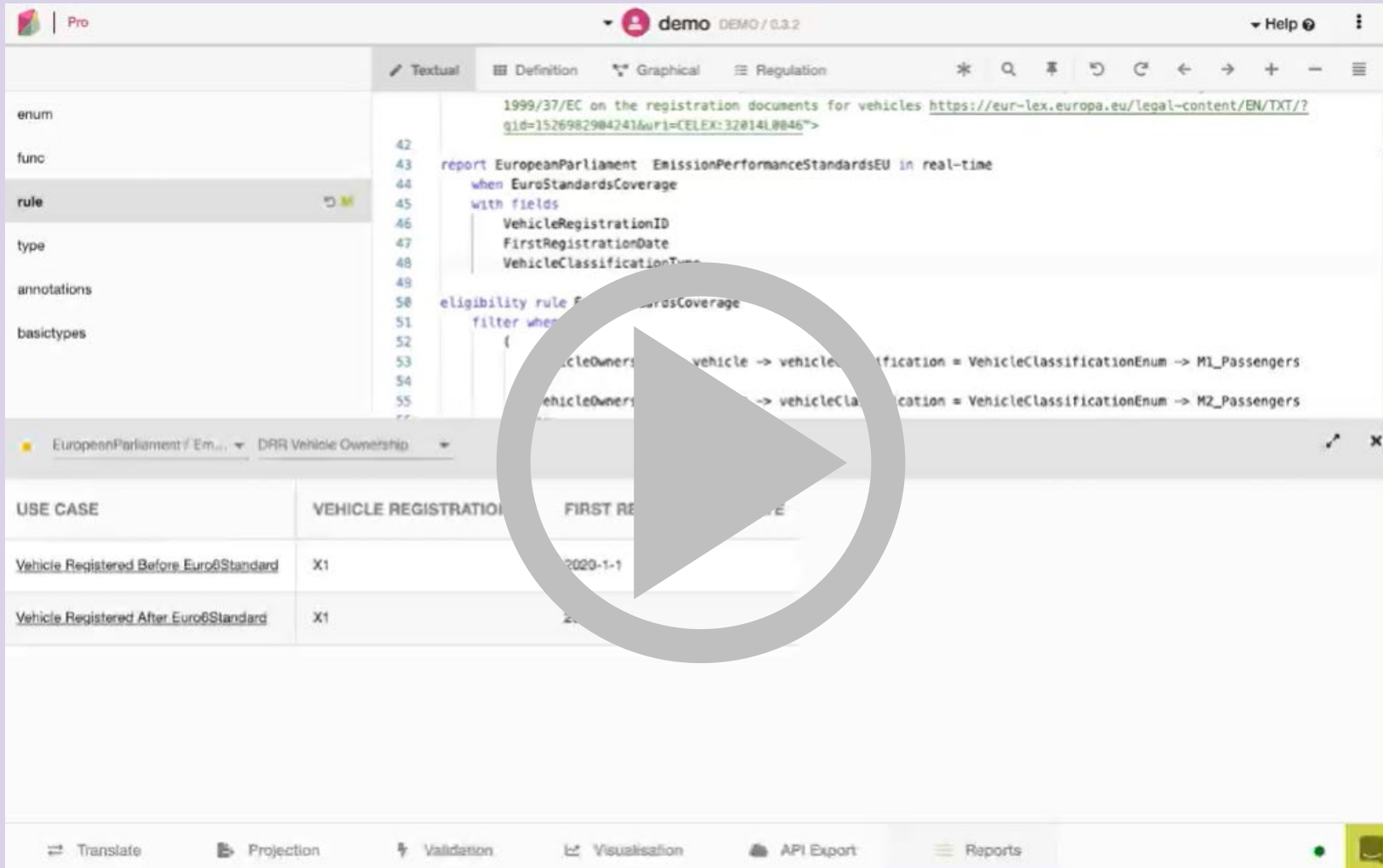
NAME	CORPUS	REFERENCE	PROVISION
<u>VehicleRegistrationID</u> Report Field	EuropeanParliament VehicleRegistrationEU	article 1	This Directive shall apply to the vehicle registration documents issued by the Member States
<u>FirstRegistrationDate</u> Report Field	EuropeanParliament VehicleRegistrationEU	article 3, section 4	In Article 3, the following paragraphs are added: 4. Member States shall record electronically data on all vehicles registered on their territory. Those data shall include: (a) all mandatory elements in accordance with point I...
<u>VehicleClassificationType</u> Report Field	EuropeanParliament VehicleRegistrationEU	article 3, section 4	In Article 3, the following paragraphs are added: 4. Member States shall record electronically data on all vehicles registered on their territory. Those data shall include: (a) all mandatory elements in accordance with point I...
<u>EngineType</u> Report Field	EuropeanParliament VehicleRegistrationEU	article 3, section 4	In Article 3, the following paragraphs are added: 4. Member States shall record electronically data on all vehicles registered on their territory. Those data shall include: (a) all mandatory elements in accordance with point I...
<u>EuroEmissionStandard</u> Report Field	EuropeanCommission StandardEmissionsEuro6	annex 1, section 8	Regulation (EC) No 715/2007 is amended as follows:
<u>EuroEmissionStandard Euro1Standard</u> Report Field	EuropeanCommission StandardEmissionsEuro1	annex 1, section 8	Amendments to the Annexes to Directive 70/220/EEC Item 5.3.1.4: The table is replaced by the following:
<u>EuroEmissionStandard Euro2Standard</u> Report Field	EuropeanCommission StandardEmissionsEuro2	annex 1, section 8	Amendments to the Annexes to Directive 70/220/EEC as amended by Directive 93/59/EEC The table in Section 5.3.1.4 is replaced by the following:
<u>EuroEmissionStandard Euro3Standard</u> Report Field	EuropeanCommission StandardEmissionsEuro3	annex 1, section 5.3.1.4	Annex I The heading reads as follows: Scope, Definitions, Application for EC Type-Approval, Granting of EC Type-Approval, Requirements and Tests, Extension of EC Type-Approval, Conformity of Production and ...
<u>EuroEmissionStandard Euro4Standard</u> Report Field	EuropeanCommission StandardEmissionsEuro4	annex 1, section 5.3.1.4	Annex I The heading reads as follows: Scope, Definitions, Application for EC Type-Approval, Granting of EC Type-Approval, Requirements and Tests, Extension of EC Type-Approval, Conformity of Production and ...

At the bottom of the interface, there is a row of icons for 'Translate', 'Projection', 'Validation', 'Visualisation', 'API Export', and 'Reports'.

Watch

- As you experienced in Module B, REGnosys provides the Rosetta platform for viewing, creating and editing data models.
- As a reminder other 3rd party applications exist in addition to Rosetta.

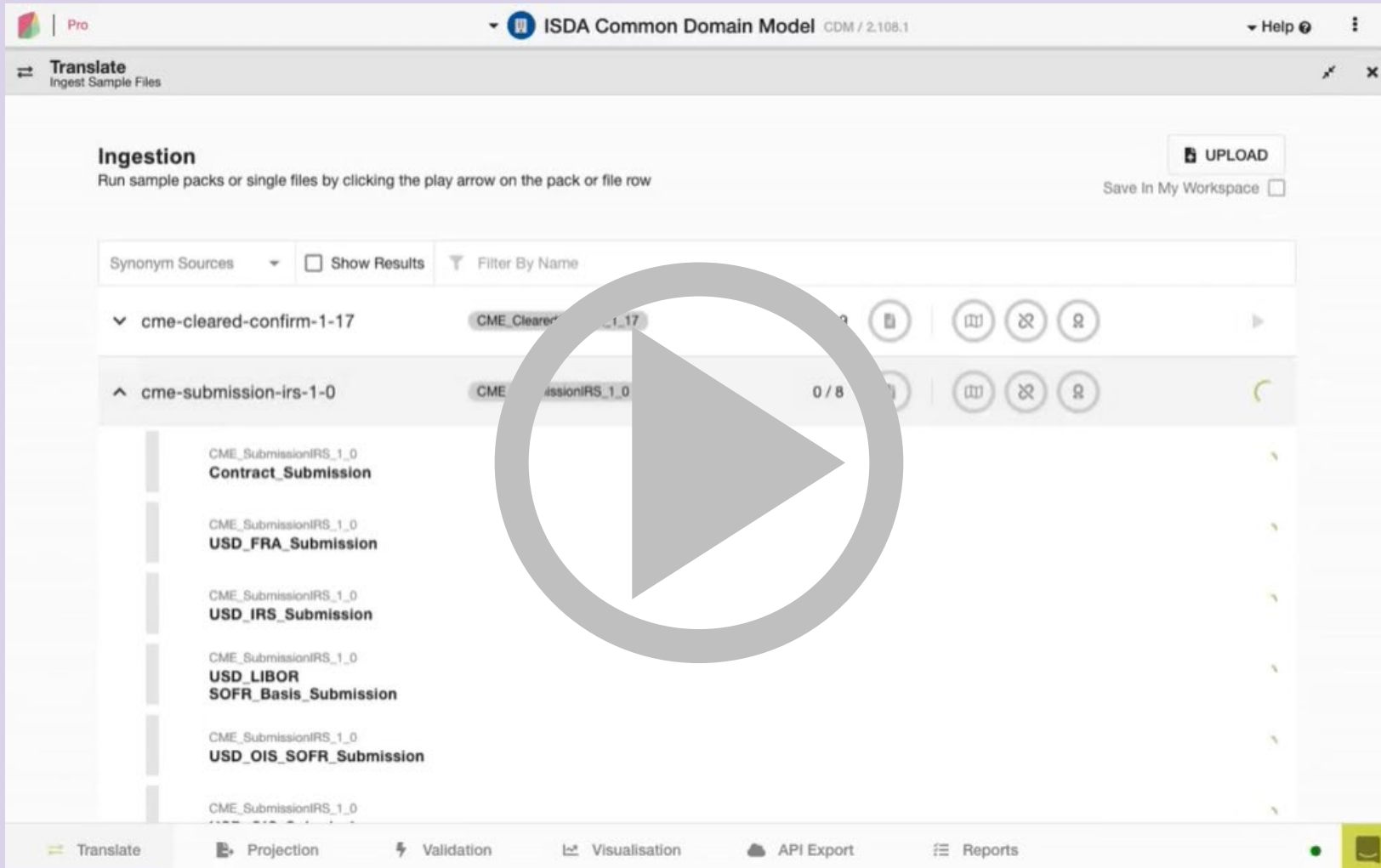
Next



Watch

- Rosetta also has additional commercial capabilities to model reporting logic and to map data using reporting rules.
- These are the core capabilities leveraged by DRR.
- As a reminder other 3rd party applications exist in addition to Rosetta.

Next



Watch

- The Rosetta Engine is also provided as Integration as a Service to accelerate adoption and provide self-service capabilities to build and maintain models and mappings.
- Visit the REGnosys website to learn more about the Rosetta platform regnosys.com
- As a reminder other 3rd party applications exist in addition to Rosetta.

Next

Rune DSL (Open Source)

Rosetta Language

- Specifies the syntax used to build model definitions

Domain Specific Language (DSL)

github.com/finos/rune-dsl

Code Generator: Java

- Automatically generates executable code based on model definitions

Other Code Generators

- Scala
- TypeScript
- DAML
- etc.

Code Generators

github.com/REGnosys/rosetta-code-generators

FINOS CDM Distribution (Open Source)

github.com/finos/common-domain-model

Model Definition

Data Model

- Product Model
- Event Model
- Legal Agreement Model

Process Model

- Calculation
- Event Logic
- Event / Product Qualification

Reporting

- Eligibility Rule
- Reportable Field
- Regulatory Reference

Data Validation

- Condition
- Special Syntax: eg choice, one-of

Annotation

- Meta-Data: eg 'key' / 'reference'
- Function Type

Namespace

- Model Partition
- Description
- Hierarchy

Interfaces

Mapping / Synonym

- FpML
- DTCC
- CME
- ISO 20022

Examples (.json)

- Product samples
- Event samples

Executable Code Distribution

Organised into namespaces

Data Model

- Product Model
- Event Model
- Legal Agreement

Process Model

- Calculation
- Event Logic
- Event / Product Qualification

Data Validation

- Condition
- Syntax: e.g. 'choice', 'one-of'

Reporting

- Eligibility Rule
- Reportable Field

Default Implementation

Hand-crafted code (not auto-generated)

Meta-Data Processor

- 'Key' hash code implementation and override
- Meta-Type
- Event Effect Post-Processor

Function

- Calculation period
- Days in period
- Resolve index

Serialiser

- JSON

Documentation cdm.finos.org

CDM Homepage

- Apache license via GitHub
- Various model definitions
- Documentation

CDM Object Builder

- Create, import, share or inspect CDM JSON
- cdm-object-builder.finos.org

Integration Test Pack

Ingestion Test (.xml, .json)

- Input samples: FpML / DTCC / CME
- Output + assertion files (JSON)

Function Test

- Function input samples (XML)
- Output + assertion files (JSON)

CDM Applications

Rosetta SDK (by REGnosys)

ui.rosetta-technology.io

Edition and Contribution

- Develop / extend the CDM
- Source control integration

Translation

- Develop / extend 'synonyms' for proprietary data models
- Manage integration test pack

Ingestion and Testing

- Convert data into CDM
- Run live integration tests
- UI or cloud-hosted API

Applications by other service providers

- We have now looked, albeit briefly, at all of the components and will drill-down further next.
- But please strengthen your understanding by reviewing this page, and the embedded links to the underlying tools, before moving on.

D

The fourth module in this series explains how CDM is used within financial services to deliver the stated benefits.

1

Overview of the CDM and the CDM components

Complete

2

Data Interfaces to the CDM and JSON

Start

3

Documentation and The Object Builder

Start

4

Introduction to Functions

Start

5

The CDM for Technologists

Start

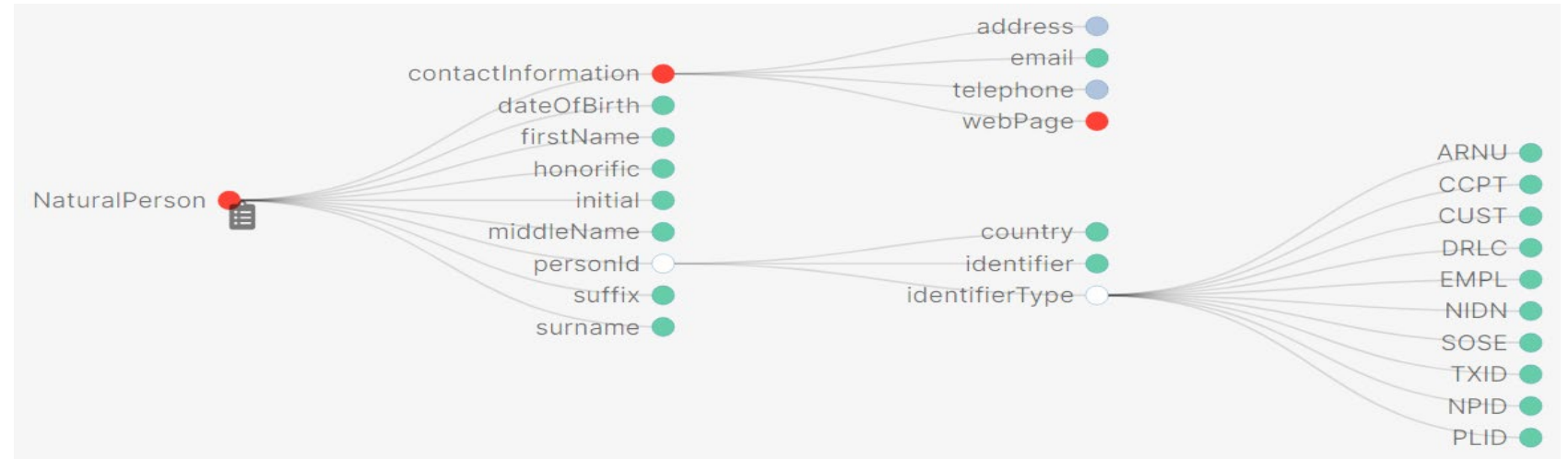
- As we have seen, the CDM is more than a data standard.
- However, to enable data transmission between different systems, we still need some standards on the use of data.
- The CDM includes support for JSON as one of the most convenient methods to represent CDM data digitally.
- What is JSON?
 - A lightweight data interchange format.
 - It is easy for humans to read and write.
 - It is easy for machines to parse and generate.

What is JSON?

Watch this short 90 second video for a quick tutorial.

[Watch](#)[Next](#)

- Let's look at an example of how this works in practice.
- Take a look at the model now and find the definition of the data type **NaturalPerson**.
- NaturalPerson** is used in the CDM to capture data about people, such as their name, date of birth and contact information.
- You can probably imagine what this data might consist of, but how would it be represented in JSON?



```

type NaturalPerson: <"A class to represent the attributes that are specific to a natural person.">
[metadata key]

personId PersonIdentifier (0..*) <"The identifier associated with a person, e.g. the internal
identification code.">
[metadata scheme]
honorific string (0..1) <"An honorific title, such as Mr., Ms., Dr. etc.">
firstName string (0..1) <"The natural person's first name. It is optional in FpML.">
middleName string (0..*) <"The natural person's middle name(s). If a middle name is provided
then an initial should be absent.">
initial string (0..*) <"The natural person's middle initial(s). If a middle initial is provided
then a name should be absent.">
surname string (0..1) <"The natural person's surname.">
suffix string (0..1) <"Name suffix, such as Jr., III, etc.">
dateOfBirth date (0..1) <"The natural person's date of birth.">
contactInformation ContactInformation (0..1) <"The contact information for such person, when
different from the contact information associated with the party.">

```

- Here is an example of the digital representation of a sample **NaturalPerson**, exported from the CDM into JSON.
- You will see that it is (fairly) readable and understandable.
- Alongside the model in the CDM, there are also tools to import and export data from the CDM to JSON.

JSON enables the swift representation of data:

```
{
  "naturalPerson": {
    "firstName": "Taylor",
    "initials": [
      "A",
      "R"
    ],
    "surname": "Swift",
    "dateOfBirth": "1989-12-13",
    "personId": {
      "country": "USA"
      "identifier": "837-12-2846"
      "identifierType": "SOSE"
    }
  }
}
```

- The notation uses curly brackets to denote objects {"...", "..."}
 
- Representation of name-value pairs and text data, ie "key": "value"
 
- Representation of a list of data items, the list is enclosed inside square brackets ["...", "..."]
 
- Representation of an object (ie **personId**) embedded inside another object (ie **naturalPerson**).
 

FINOS CDM Distribution (Open Source)
github.com/finos/common-domain-model**Model Definition****Data Model**

- Product Model
- Event Model
- Legal Agreement Model

Process Model

- Calculation
- Event Logic
- Event / Product Qualification

Reporting

- Eligibility Rule
- Reportable Field
- Regulatory Reference

Data Validation

- Condition
- Special Syntax: eg choice, one-of

Annotation

- Meta-Data: eg 'key' / 'reference'
- Function Type

Namespace

- Model Partition
- Description
- Hierarchy

Interfaces**Mapping / Synonym**

- FpML
- DTCC
- CME
- ISO 20022

Examples (.json)

- Product samples
- Event samples

- Now that you understand JSON, let's look at some real examples of product data modelled using the CDM and represented in JSON that have been contributed by members.
- Visit the following page on GitHub:
<https://github.com/finos/common-domain-model/tree/master/rosetta-source/src/main/resources/result-json-files>
- Browse this site to navigate the samples including:
 - CME cleared OTC contracts
 - DTCC trade reporting
 - FpML transactions
- We will take a more detailed look at collateral-specific JSON samples, and how you can both read and manipulate them, in the next module.
- The capabilities used to produce these samples are also the cornerstone of ISDA's Digital Regulatory Reporting initiative.

D

The fourth module in this series explains how CDM is used within financial services to deliver the stated benefits.

1

Overview of the CDM and the CDM components

Complete

2

Data Interfaces to the CDM and JSON

Complete

3

Documentation and The Object Builder

Start

4

Introduction to Functions

Start

5

The CDM for Technologists

Start

FINOS CDM Distribution (Open Source)

github.com/finos/common-domain-model

Model Definition

Data Model

- Product Model
- Event Model
- Legal Agreement Model

Process Model

- Calculation
- Event Logic
- Event / Product Qualification

Reporting

- Eligibility Rule
- Reportable Field
- Regulatory Reference

Data Validation

- Condition
- Special Syntax: eg choice, one-of

Annotation

- Meta-Data: eg 'key' / 'reference'
- Function Type

Namespace

- Model Partition
- Description
- Hierarchy

Interfaces

Mapping / Synonym

- FpML
- DTCC
- CME
- ISO 20022

Examples (.json)

- Product samples
- Event samples

Executable Code Distribution

Organised into namespaces

Data Model

- Product Model
- Event Model
- Legal Agreement

Process Model

- Calculation
- Event Logic
- Event / Product Qualification

Data Validation

- Condition
- Syntax: e.g. 'choice', 'one-of'

Reporting

- Eligibility Rule
- Reportable Field

Default Implementation

Hand-crafted code (not auto-generated)

Meta-Data

- Processor
- 'Key' hash code implementation and override
- Meta-Type
- Event Effect Post-Processor

Function

- Calculation period
- Days in period
- Resolve index

Serialiser

- JSON

Documentation cdm.finos.org

CDM Homepage

- Apache license via GitHub
- Various model definitions
- Documentation

CDM Object Builder

- Create, import, share or inspect CDM JSON
- cdm-object-builder.finos.org

Integration Test Pack

Ingestion Test (.xml, .json)

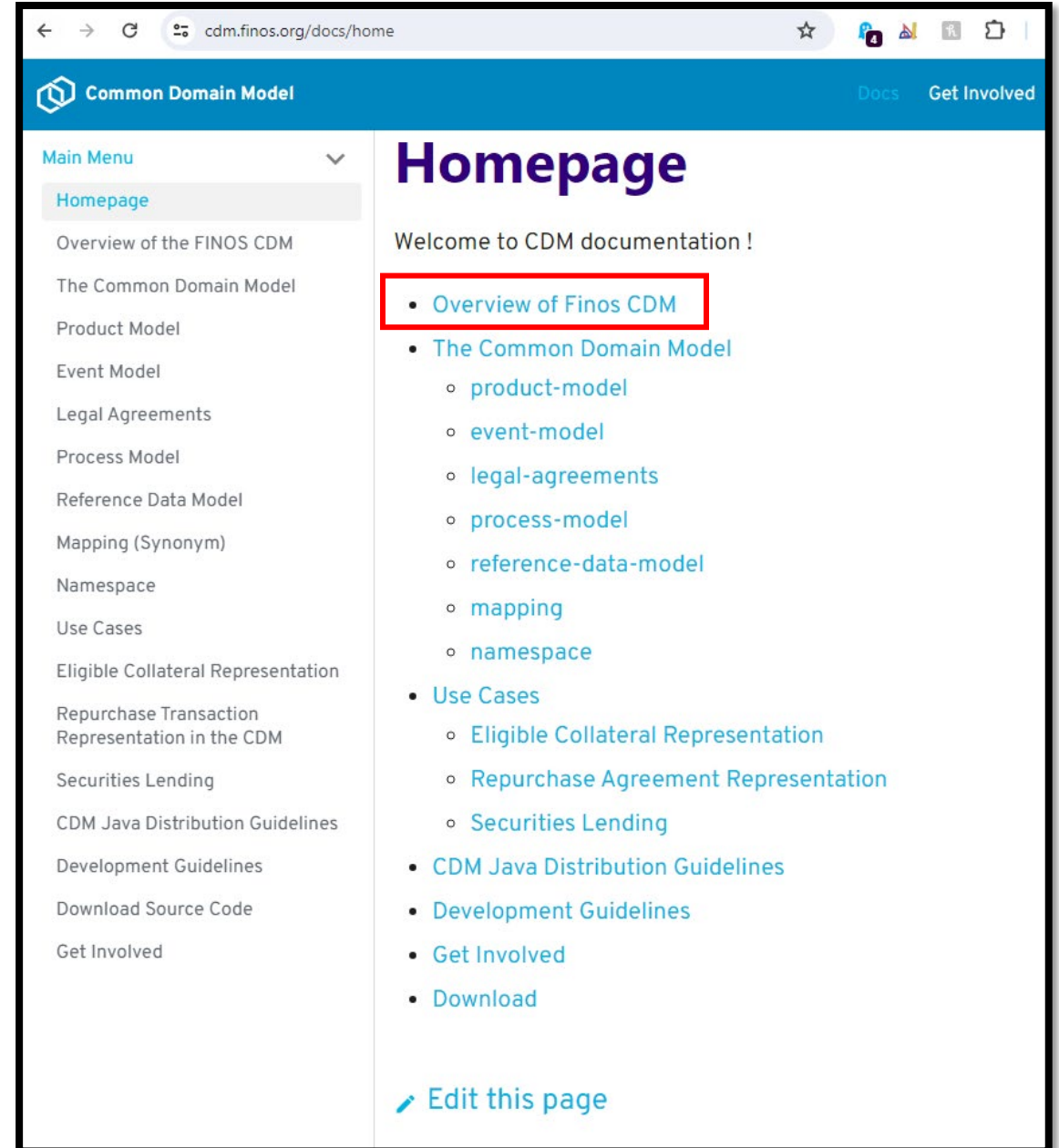
- Input samples: FpML / DTCC / CME
- Output + assertion files (JSON)

Function Test

- Function input samples (XML)
- Output + assertion files (JSON)

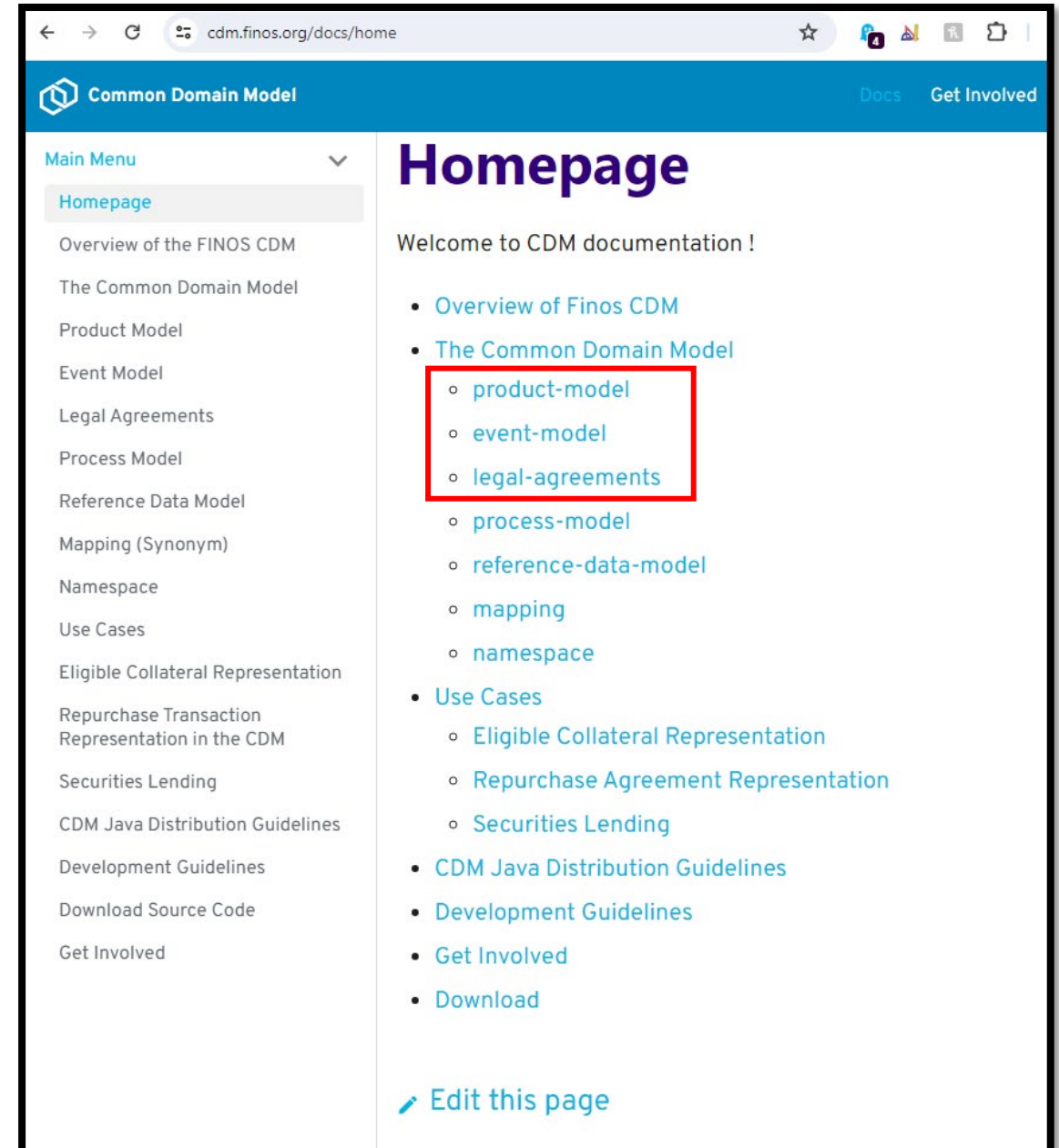
- Hopefully you are now gaining an understanding of the breadth of capabilities in the CDM and the features provided in the FINOS CDM Distribution.
- There is also incredible depth, far beyond what we can cover in this Get Started Guide. But there are two resources which we would like to highlight:
 - Documentation** – including on the CDM Model Definition but also on the Rosetta DSL and Rosetta Platform.
 - The **CDM Object Builder** – a tool built specifically for the Collateral community to help understand the CDM Model.

- The CDM Homepage on FINOS is the main source of documentation.
- Please visit the page now and read the section [“Overview of FINOS CDM”](#).

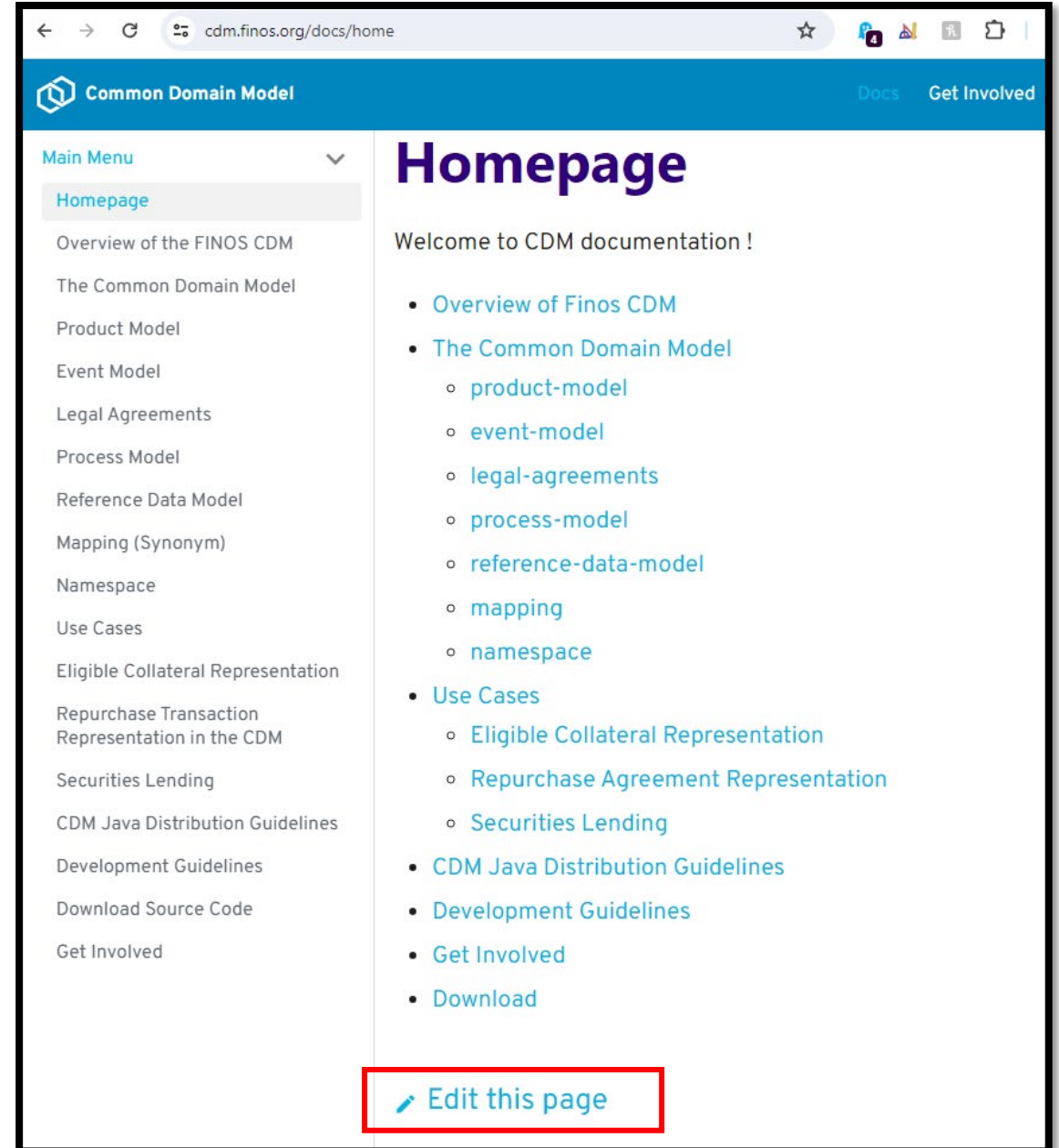
[Next](#)

The screenshot shows the CDM Homepage on the FINOS website. The browser address bar displays `cdm.finos.org/docs/home`. The page has a blue header with the CDM logo and the text "Common Domain Model". On the right side of the header are links for "Docs" and "Get Involved". A left sidebar titled "Main Menu" contains a list of navigation items, with "Homepage" highlighted. The main content area is titled "Homepage" and includes a welcome message: "Welcome to CDM documentation !". Below this, a list of links is displayed, with "Overview of Finos CDM" highlighted by a red rectangle. Other links include "The Common Domain Model" (with sub-links for product-model, event-model, legal-agreements, process-model, reference-data-model, mapping, and namespace), "Use Cases" (with sub-links for Eligible Collateral Representation, Repurchase Agreement Representation, and Securities Lending), "CDM Java Distribution Guidelines", "Development Guidelines", "Get Involved", and "Download". At the bottom right, there is a link to "Edit this page" with a pencil icon.

- The CDM Homepage on FINOS is the main source of documentation.
- If you would like to further extend your learning, beyond the previous module, we suggest that you look at the next section “The Common Domain Model” and particularly
 - [product-model](#)
 - [event-model](#)
 - [legal-agreements](#).
- Hold off on reading “Use Cases” and the following technical topics until we get there!

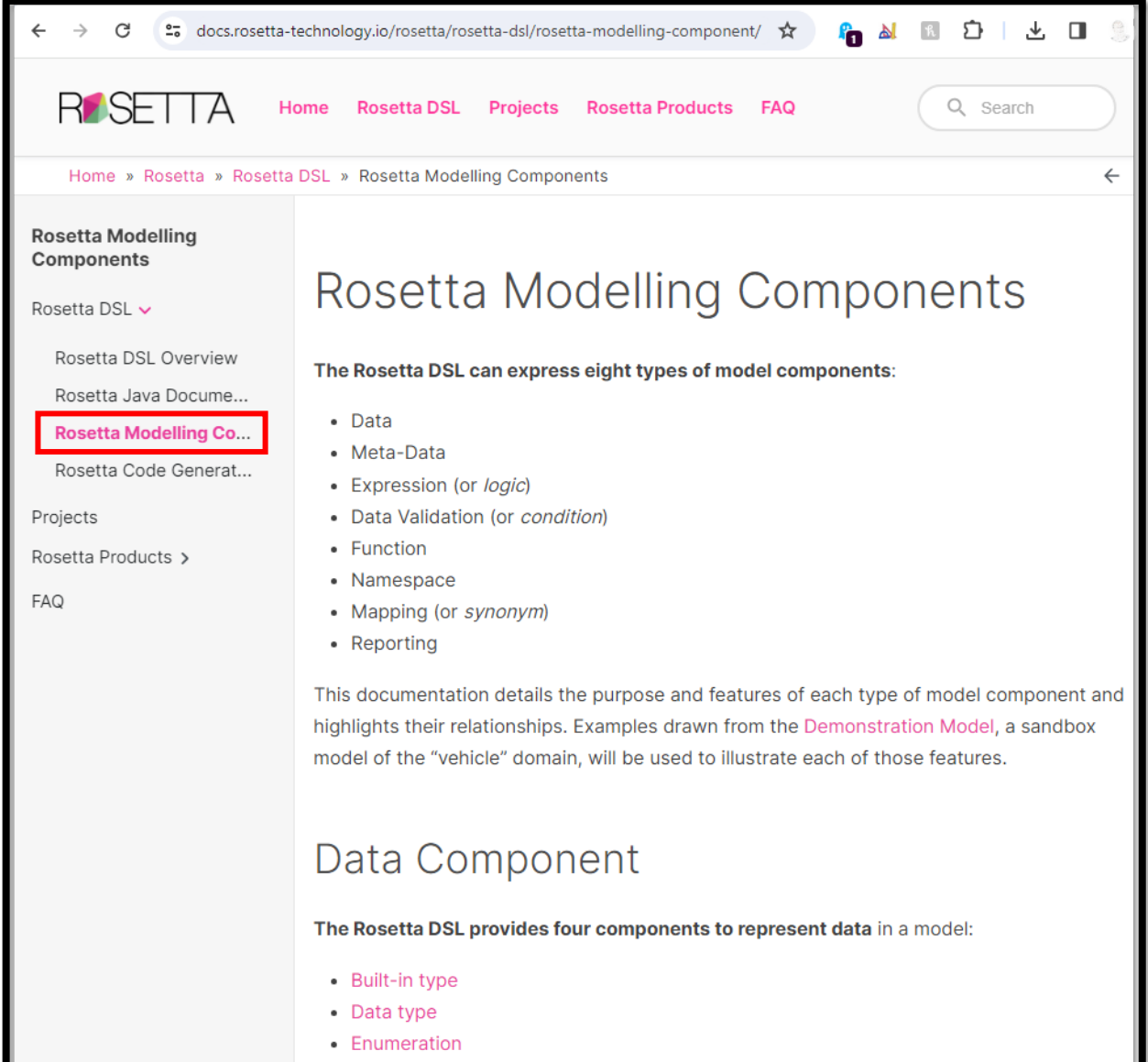
[Next](#)

- By the way, did you see anything that wasn't quite right in the documentation?
- There are always cases where documentation lags the actual product and sometimes mistakes slip through!
- But this Documentation is also open source, and published on GitHub.
- So you, or indeed anyone, can contribute a correction or enhancement and all contributions which aid the community's understanding and adoption are warmly welcomed.

[Next](#)

The screenshot shows the homepage of the Common Domain Model (CDM) documentation. The browser address bar displays `cdm.finos.org/docs/home`. The page has a blue header with the CDM logo and the text "Common Domain Model". On the right side of the header are links for "Docs" and "Get Involved". A left sidebar titled "Main Menu" contains a list of topics: "Homepage" (highlighted), "Overview of the FINOS CDM", "The Common Domain Model", "Product Model", "Event Model", "Legal Agreements", "Process Model", "Reference Data Model", "Mapping (Synonym)", "Namespace", "Use Cases", "Eligible Collateral Representation", "Repurchase Transaction Representation in the CDM", "Securities Lending", "CDM Java Distribution Guidelines", "Development Guidelines", "Download Source Code", and "Get Involved". The main content area is titled "Homepage" and says "Welcome to CDM documentation !". It features a bulleted list of links: "Overview of Finos CDM", "The Common Domain Model" (with sub-links for product-model, event-model, legal-agreements, process-model, reference-data-model, mapping, and namespace), "Use Cases" (with sub-links for Eligible Collateral Representation, Repurchase Agreement Representation, and Securities Lending), "CDM Java Distribution Guidelines", "Development Guidelines", "Get Involved", and "Download". At the bottom right, a red-bordered button with a pencil icon and the text "Edit this page" is visible.

- We have given you an overview of the Rune Domain-Specific Language (DSL) that supports the modelling of operational processes in the CDM.
- You can find further details of the use and syntax of the DSL on the [Rune DSL documentation](https://docs.rosetta-technology.io/rosetta/rosetta-dsl/rosetta-modelling-component/) website.
- If you intend to participate in detailed work with CDM, or to contribute your own changes, we recommend that you read and bookmark this documentation page.

[Next](#)

The screenshot shows a web browser displaying the Rosetta DSL documentation page for Rosetta Modelling Components. The URL in the address bar is `docs.rosetta-technology.io/rosetta/rosetta-dsl/rosetta-modelling-component/`. The page features a navigation bar with links to Home, Rosetta DSL, Projects, Rosetta Products, and FAQ. A search bar is also present. The main content area is titled "Rosetta Modelling Components" and lists eight types of model components: Data, Meta-Data, Expression (or *logic*), Data Validation (or *condition*), Function, Namespace, Mapping (or *synonym*), and Reporting. Below this, a section titled "Data Component" lists four components: Built-in type, Data type, and Enumeration. The left sidebar contains a list of links, with "Rosetta Modelling Co..." highlighted in a red box.

docs.rosetta-technology.io/rosetta/rosetta-dsl/rosetta-modelling-component/

ROSETTA Home Rosetta DSL Projects Rosetta Products FAQ

Home » Rosetta » Rosetta DSL » Rosetta Modelling Components

Rosetta Modelling Components

Rosetta DSL ▾

- Rosetta DSL Overview
- Rosetta Java Docume...
- Rosetta Modelling Co...**
- Rosetta Code Generat...

Projects

Rosetta Products >

FAQ

Rosetta Modelling Components

The Rosetta DSL can express eight types of model components:

- Data
- Meta-Data
- Expression (or *logic*)
- Data Validation (or *condition*)
- Function
- Namespace
- Mapping (or *synonym*)
- Reporting

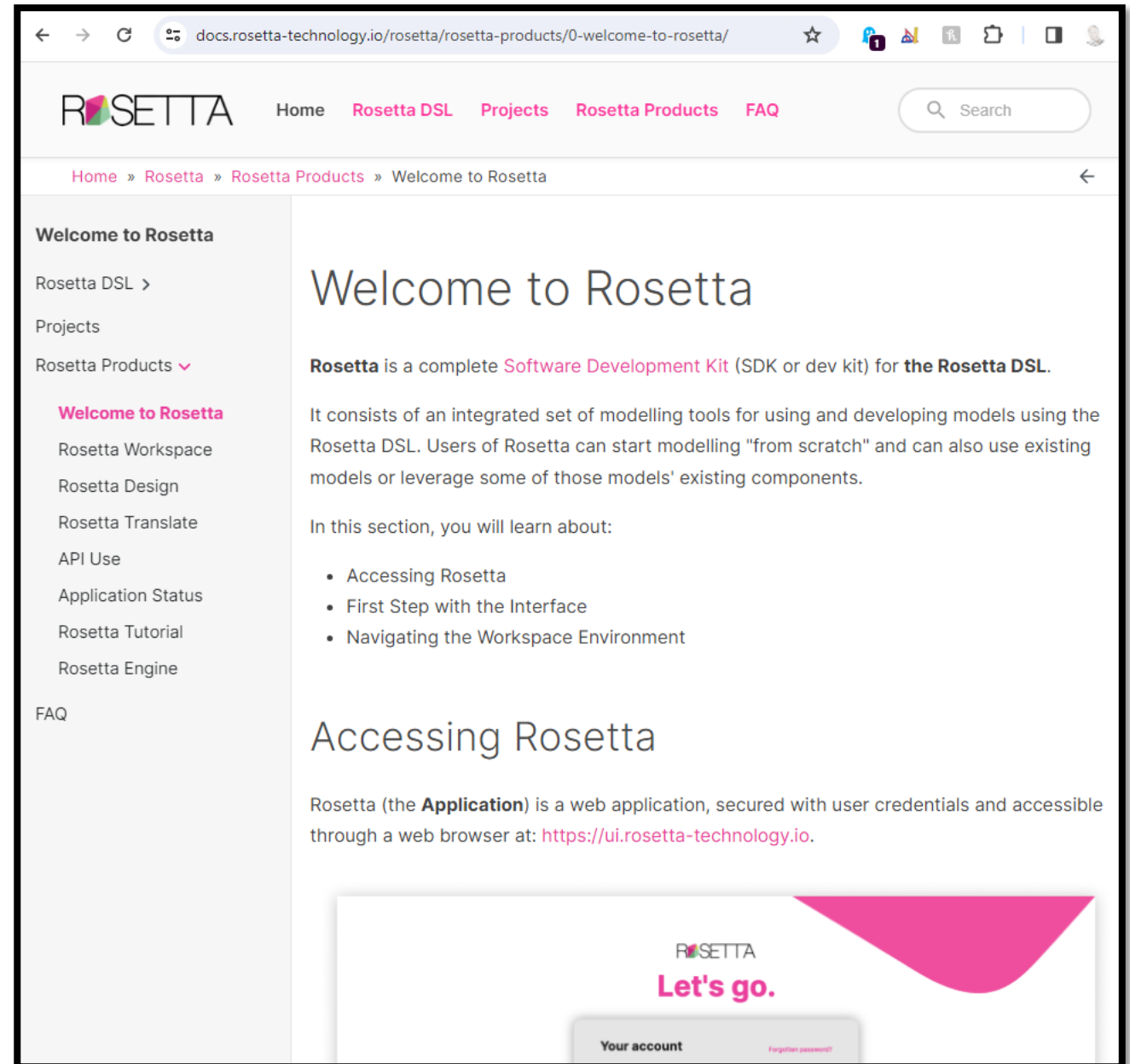
This documentation details the purpose and features of each type of model component and highlights their relationships. Examples drawn from the **Demonstration Model**, a sandbox model of the "vehicle" domain, will be used to illustrate each of those features.

Data Component

The Rosetta DSL provides four components to represent data in a model:

- Built-in type
- Data type
- Enumeration

- Study the [Rosetta Platform documentation](#) website for further information about these products:
- [Rosetta Workspace](#) – a user area that allows the user to work with a model in a "safe space".
- [Rosetta Design](#) – a sophisticated model editor that allows the user to edit model files in their own Workspace.
- [Rosetta Translate](#) – to keep in control of a mapping layer between existing systems, databases and message formats and the CDM.
- [Rosetta Engine](#) – run the functional logic contained in a model on physical data.

[Next](#)

- The Object Builder is a useful CDM tool that can be used to manipulate JSON data within the CDM.
- Using the Object Builder, you can create, import, share, inspect or export data to and from the CDM using JSON; the data can also be viewed in a tabular format.

Object Builder for the CDM™

CDM-JAVA 6.0.0

Builder

Viewer

Select root type

+

EligibleCollateralSpecifi...

Represents a set of criteria used to specify eligible collateral.

Import

Export

Clear

EligibleCollateralSpecification -> criteria

+

criteria

EligibleCollateralCriteria (1..*)

+

collateralCriteria

CollateralCriteria (1..1)

+

AllCriteria

AllCriteria (0..1)

+

allCriteria

CollateralCriteria (2..*)

>

+

AssetType

AssetType (0..1)

+

allCriteria

CollateralCriteria (2..*)

+

AssetMaturity

AssetMaturity (0..1)

+

maturityRange

PeriodRange (1..1)

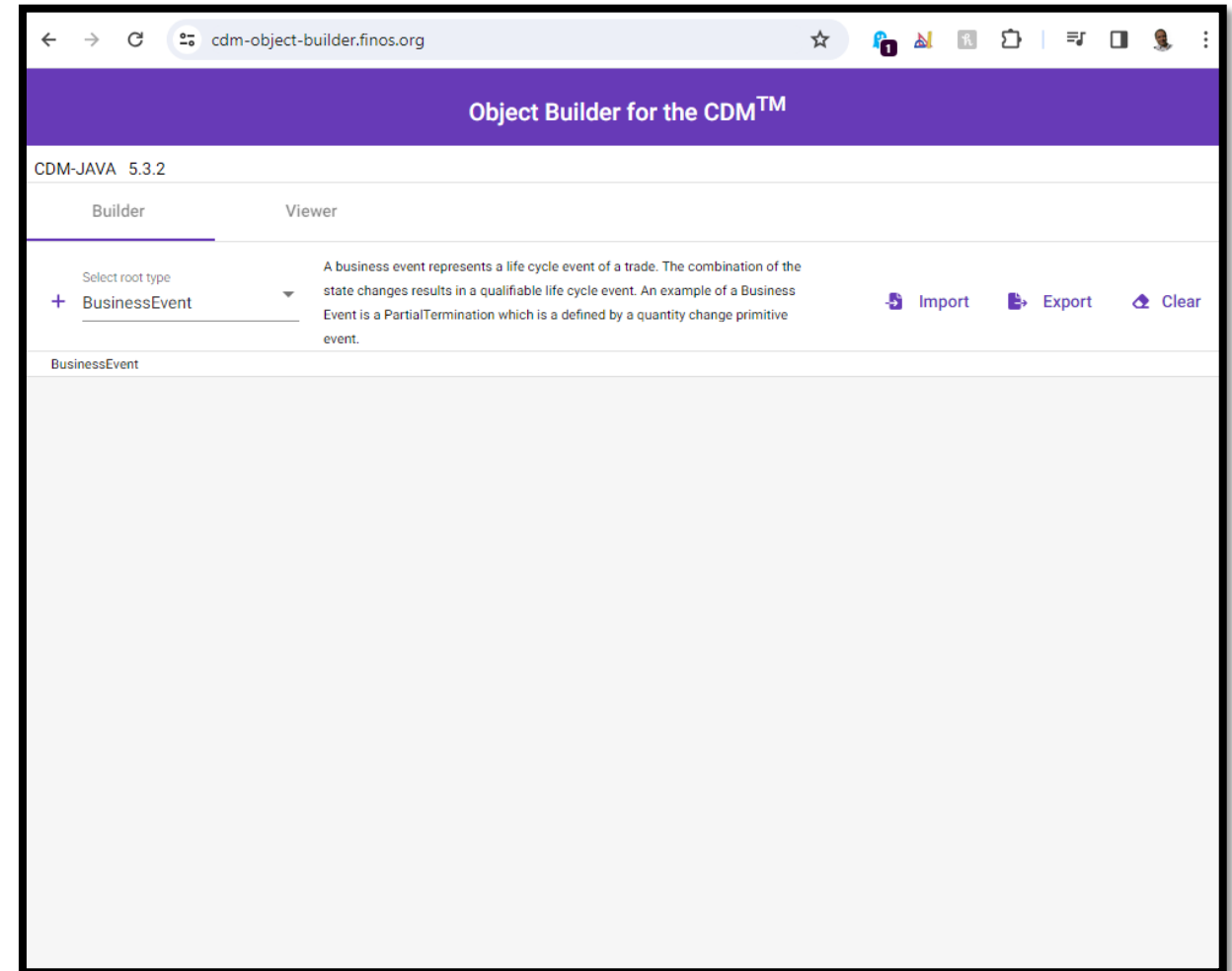
criteria	collateralCriteria	AllCriteria	allCriteria 1	AssetType	assetType	
					securityType	
			allCriteria 2	AssetMaturity	maturityRange	lowerBound
						period
						periodMultiplier
						upperBound
						period
						periodMultiplier
					maturityType	
			allCriteria 3	IssuerCountryOfOrigin	issuerCountryOfOrigin	
			allCriteria 4	CollateralIssuerType	issuerType	
	treatment				isIncluded	
		valuationTreatment			haircutPercentage	

```

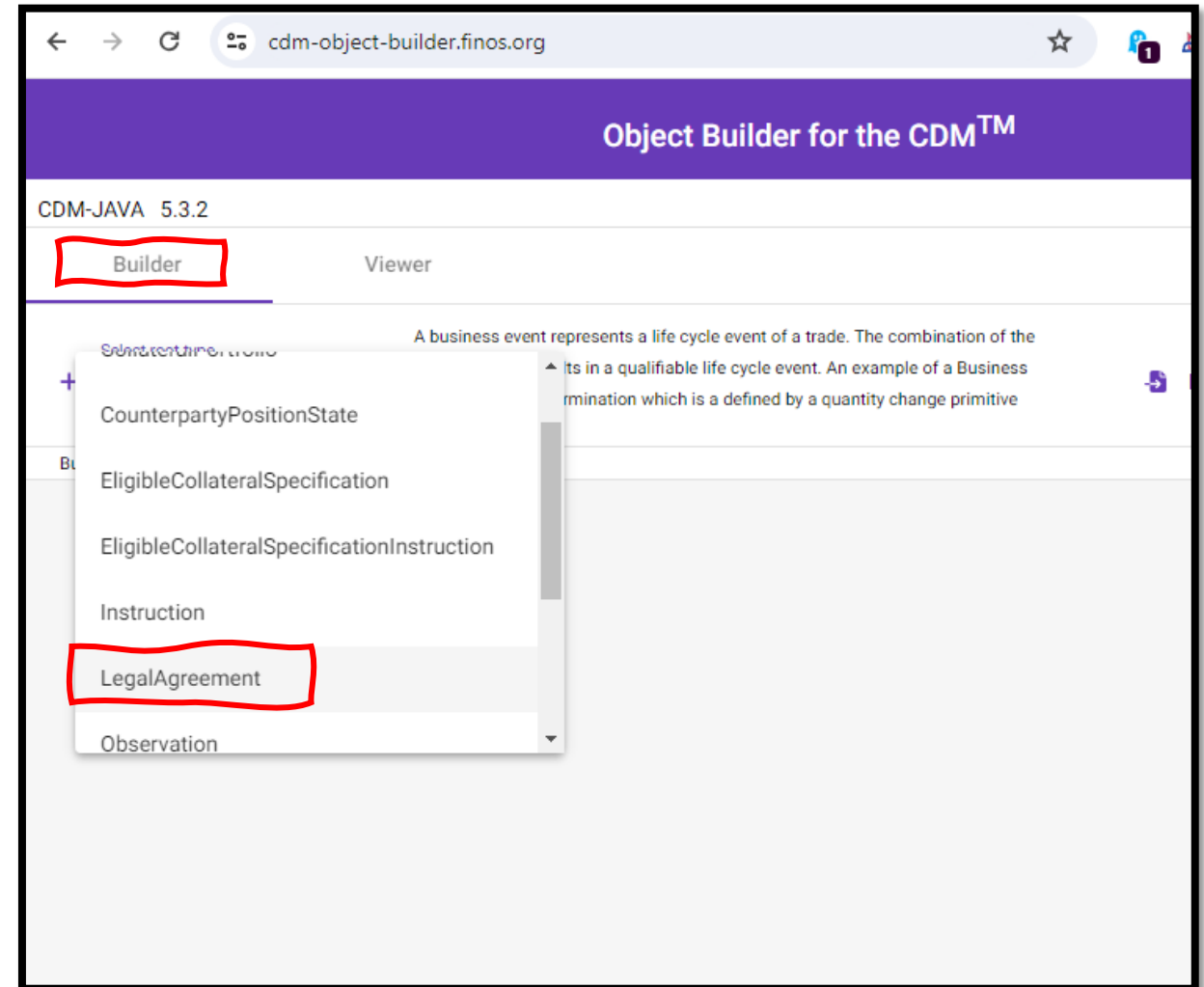
    },
    "maturityType": "REMAINING_MATURITY"
  },
  {
    "IssuerCountryOfOrigin": {
      "issuerCountryOfOrigin": "US"
    },
    "CollateralIssuerType": {
      "issuerType": "SOVEREIGN_CENTRAL_BANK"
    }
  },
  "treatment": {
    "isIncluded": true,
    "valuationTreatment": {
      "haircutPercentage": 0.05
    }
  }
}

```

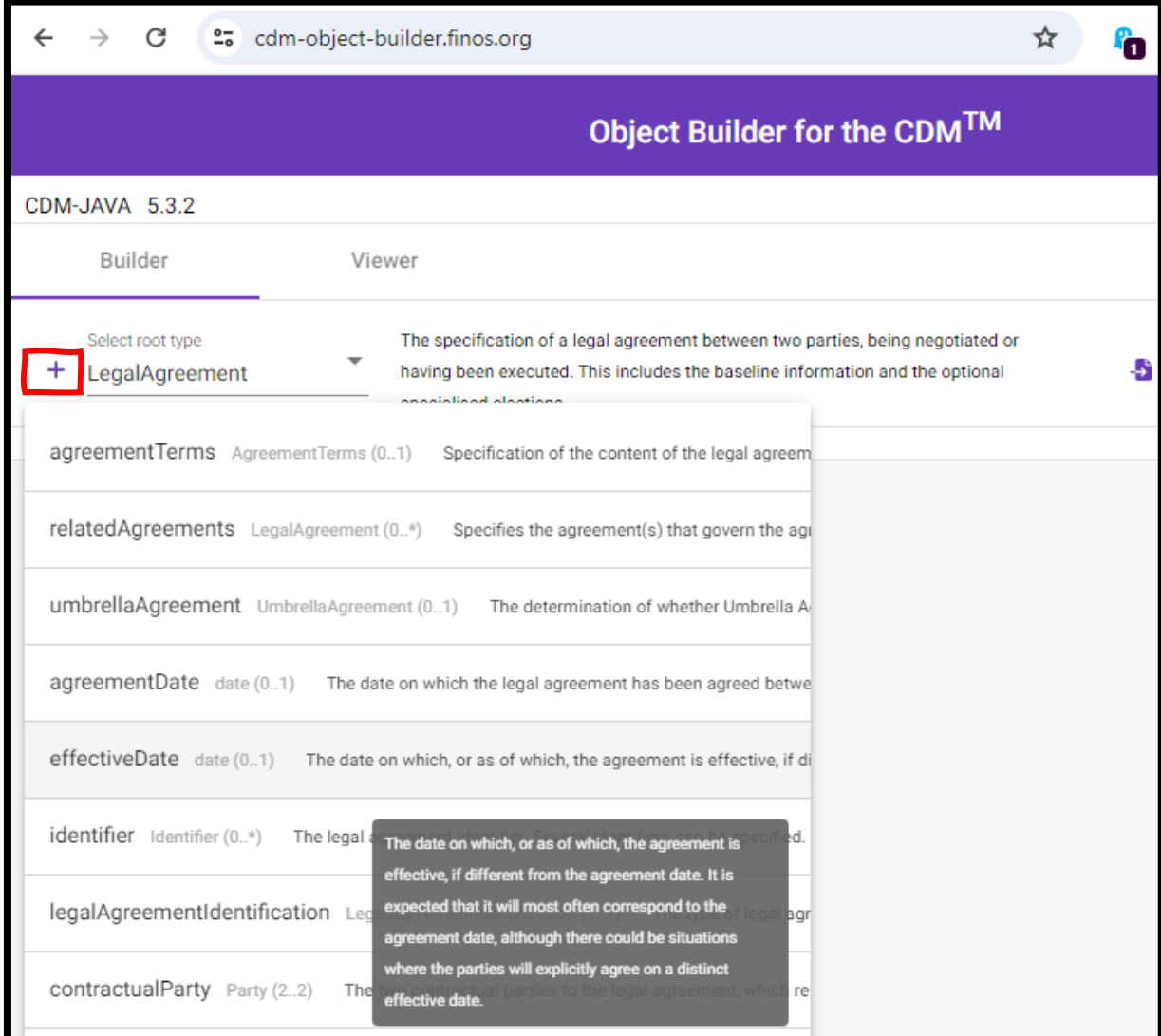
- The **Object Builder** tool is also available open source on FINOS, although it is not part of the CDM Distribution.
- The code can be found here:
<https://github.com/finos/cdm-object-builder>
- Market participants can copy the code and use it internally, including building the functionality into their own workflows and applications.
- The **Object Builder** is also available as a service hosted by FINOS and is openly accessible for use here:
<https://cdm-object-builder.finos.org/>
- Please open the **Object Builder** service now, using the second link above.



- Let's see how the **Object Builder** can be used to create an object in the CDM.
- The **Object Builder** has two main tabs:
 - The **Builder** tab: used to create and update data.
 - The **Viewer** tab: used to view data in both table and code (JSON) form.
- In the Builder tab, data can be created for any of the root data types in the CDM.
- Using the dropdown, find and select “LegalAgreement” and we will use this to create an example *ISDA Credit Support Agreement*.



- Once the “LegalAgreement” is selected as the root type, click on the + symbol and a list of the attributes for the data type are displayed, along with their definitions and descriptions.
- This information is retrieved directly from the CDM (accessing the production version of the model, 5.3.2 in this case).
- Let’s create a hypothetical ISDA Credit Support Agreement with the following terms:
 - Based on the *ISDA 2018 Credit Support Annex*.
 - Between counterparties *BigBank* and *GlobalFund*.
 - Agreed on 29 February and effective 3 days later.



The screenshot shows the "Object Builder for the CDM™" web application. The browser address bar displays "cdm-object-builder.finos.org". The application has a purple header bar with the title "Object Builder for the CDM™". Below the header, it shows "CDM-JAVA 5.3.2". There are two tabs: "Builder" (selected) and "Viewer". In the "Builder" tab, there is a "Select root type" dropdown menu. The "LegalAgreement" option is selected and highlighted with a red box. A red "+" icon is visible next to the "LegalAgreement" text. Below the dropdown, a list of attributes is displayed, each with its name, multiplicity, and description. The attributes listed are:

- agreementTerms**: AgreementTerms (0..1) - Specification of the content of the legal agreement
- relatedAgreements**: LegalAgreement (0..*) - Specifies the agreement(s) that govern the agreement
- umbrellaAgreement**: UmbrellaAgreement (0..1) - The determination of whether Umbrella Agreement
- agreementDate**: date (0..1) - The date on which the legal agreement has been agreed between
- effectiveDate**: date (0..1) - The date on which, or as of which, the agreement is effective, if different from the agreement date. It is expected that it will most often correspond to the agreement date, although there could be situations where the parties will explicitly agree on a distinct effective date.
- identifier**: Identifier (0..*) - The legal agreement identifier
- legalAgreementIdentification**: LegalAgreementIdentification (0..1) - The legal agreement identification
- contractualParty**: Party (2..2) - The contractual parties to the legal agreement, which represent the legal entities

- Follow these steps to enter the data:
 - Select **legalAgreementIdentification** and then **agreementName** and **agreementType**; choose the appropriate value from the dropdown. Then add the **creditSupportAgreementType**.
 - From the top level, add the **agreementDate** and **effectiveDate**.
 - Continue you to select attributes from the **LegalAgreement** that match our use case and add them to the object as you see fit.
 - You can try out different features of the model to add complexity and specificity.

Select root type
 + LegalAgreement

The specification of a legal agreement between two parties, being negotiated or having been executed. This includes the baseline information and the optional specialised elections

LegalAgreement

agreementDate	date (0..1)	2024-02-29	
effectiveDate	date (0..1)	2024-03-05	
+ legalAgreementIdentification LegalAgreementIdentification (1..1)			
+ agreementName AgreementName (1..1)			
agreementType	LegalAgreementTypeEnum (1..1)	CreditSupportAgreement	
creditSupportAgreementType	CreditSupportAgreementTypeEnum (0..1)	CreditSupportAnnex	

Object Builder for the CDM™

CDM-JAVA 5.3.2

Builder

Viewer

Tabular

JSON

☐ Transpose

agreementDate			2024-02-29
agreementTerms	agreement		
contractualParty 1	partyId	identifier	BANKUS33XXX
		identifierType	BIC
contractualParty 2	partyId	identifier	425900GLBFUND0PN55Z92
		identifierType	LEI
effectiveDate			2024-03-05
identifier	assignedIdentifier	identifier	CSA001
legalAgreementIdentification	agreementName	agreementType	CREDIT_SUPPORT_AGREEMENT
		creditSupportAgreementType	CREDIT_SUPPORT_ANNEX
	governingLaw		USNY
	publisher		ISDA
	vintage		2018

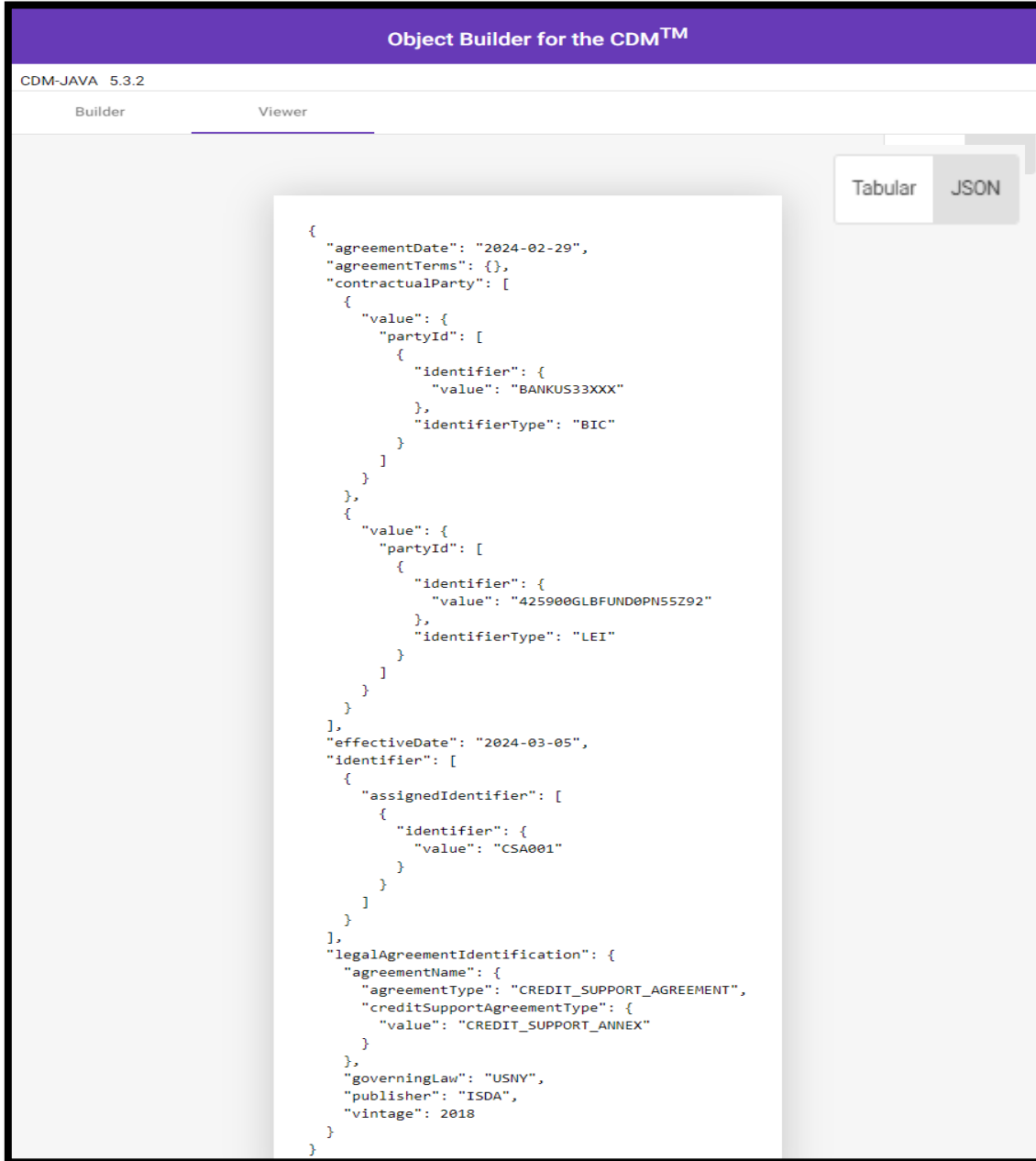
- When you have completed entering all of the data – or indeed at any point during the process – you can click on the Viewer tab to look at the CDM objects that have been created.

- The CDM objects can be viewed in tabular form horizontally (or transposed vertically)...

[View Table](#)

- ... or the CDM objects can be serialised into JSON.

[View JSON](#)



The screenshot shows the 'Object Builder for the CDM™' application. The top bar is purple with the title. Below it, a white bar shows 'CDM-JAVA 5.3.2' and two tabs: 'Builder' and 'Viewer'. The 'Viewer' tab is active. On the right, there are two sub-tabs: 'Tabular' and 'JSON', with 'JSON' selected. The main area displays a JSON object representing a CDM object. The JSON structure includes fields like 'agreementDate', 'agreementTerms', 'contractualParty', 'effectiveDate', 'identifier', 'legalAgreementIdentification', 'governingLaw', 'publisher', and 'vintage'.

```

{
  "agreementDate": "2024-02-29",
  "agreementTerms": {},
  "contractualParty": [
    {
      "value": {
        "partyId": [
          {
            "identifier": {
              "value": "BANKUS33XXX"
            },
            "identifierType": "BIC"
          }
        ]
      }
    },
    {
      "value": {
        "partyId": [
          {
            "identifier": {
              "value": "425900GLBFUND0PN55Z92"
            },
            "identifierType": "LEI"
          }
        ]
      }
    }
  ],
  "effectiveDate": "2024-03-05",
  "identifier": [
    {
      "assignedIdentifier": [
        {
          "identifier": {
            "value": "CSA001"
          }
        }
      ]
    }
  ],
  "legalAgreementIdentification": {
    "agreementName": {
      "agreementType": "CREDIT_SUPPORT_AGREEMENT",
      "creditSupportAgreementType": {
        "value": "CREDIT_SUPPORT_ANNEX"
      }
    }
  },
  "governingLaw": "USNY",
  "publisher": "ISDA",
  "vintage": 2018
}
  
```

- When you have completed entering all of the data – or indeed at any point during the process – you can click on the Viewer tab to look at the CDM objects that have been created.

- The CDM objects can be viewed in tabular form horizontally (or transposed vertically)...

View table

- ... or the CDM objects can be serialised into JSON.

View JSON

Next

- Back on the **Builder** tab, the completed objects can be exported to a file, that is, as JSON data.
- It is also possible to import JSON files into the Object Builder as well.
- This can be used to share data, validate information, support testing, etc.
- We will use these features again in the next module when we will look at some specific Collateral use cases.

Object Builder for the CDM™

CDM-JAVA 5.3.2

Builder

Viewer

Select root type

+ LegalAgreement

The specification of a legal agreement between two parties, being negotiated or having been executed. This includes the baseline information and the optional specialised elections

Import

Export

Clear

LegalAgreement

agreementDate	date (0..1)	2024-02-29		
> +	agreementTerms	AgreementTerms (0..1)	:	
> +	contractualParty	Party (2..2)	:	
> +	contractualParty	Party (2..2)	:	
effectiveDate	date (0..1)	2024-03-05		

D

The fourth module in this series explains how CDM is used within financial services to deliver the stated benefits.

1

Overview of the CDM and the CDM components

Complete

2

Data Interfaces to the CDM and JSON

Complete

3

Documentation and The Object Builder

Complete

4

Introduction to Functions

Start

5

The CDM for Technologists

Start

- Continuing our exploration into the key components within the CDM, we will now look at *functions*.
- A *function* is one of the features in CDM which delivers automation and APIs, not just a data model.
- *Functions* can take inputs, perform processing or calculations, and return outputs.
- *Functions* are specifically used for event processing – that is, updating the state of an item (e.g. a trade) given a processing instruction.
- This delivers the power of the CDM to standardize processing and operational workflow across the industry, not only defining data standards.

- Like data types, functions are written in the CDM using the Rosetta DSL; here is an example of a simple function which identifies which of two numbers is the larger:

```
func Max: <"Returns the greater number of two supplied numbers.">
  inputs:
    a number (1..1)
    b number (1..1)
  output:
    result number (1..1)
  set result: if a > b then a else b
```

- The function “Max” sets the output “result” to the larger of the two supplied inputs, “a” and “b”.
- “Max” can be found in the `cdm.base.math:func` namespace; have a look at it now, and the other related functions for date manipulation (in `cdm.base.datetime:func`) and daycount processing (in `cdm.base.datetime.daycount:func`).

- In the CDM, the type of a particular asset or product (e.g. Interest Rate Swap, Commodity, etc) is inferred from the economic terms of the product instead of by explicitly naming the product type.
- The CDM uses a set of **Product Qualification functions** to achieve this purpose.
- These functions have the same format as other ones but are additionally identified with a **[qualification Product]** annotation to signify their specific role and behaviour.
- The output of the qualification function is used to populate the **productQualifier** attribute of the **ProductTaxonomy** object, which is created when a **ContractualProduct** object is created.

- Here is the function that qualifies a specific product as being “Foreign Exchange”; for all of the product qualification functions, the input is the **EconomicTerms** of the product and the output is a **Boolean** yes/no answer:

```
func Qualify_AssetClass_ForeignExchange: <"Qualifies a product as having the Asset Class
classification Foreign Exchange">
  inputs:
    economicTerms EconomicTerms (1..1)
  output:
    is_product boolean (1..1)

  set is_product:
    economicTerms -> payout -> forwardPayout -> underlier -> foreignExchange exists
    or economicTerms -> payout -> optionPayout -> underlier -> foreignExchange exists
    or economicTerms -> payout -> performancePayout -> observationTerms -> observable
    -> currencyPair exists
```

- The CDM implements the ISDA Product Taxonomy v2.0 to qualify contractual products, foreign exchange, and repurchase agreements.

- The CDM supports Product Qualification functions for Credit Derivatives, Interest Rate Derivatives, Equity Derivatives, Foreign Exchange, and Repurchase Agreements.
- A Product Qualification function applies a taxonomy-specific business logic to identify if the product attribute values, as represented by the product's economic terms, match the specified criteria for the product named in that taxonomy. For example, if a certain set of attributes are populated and others are absent, then that specific product type is inferred.
- In a typical CDM model implementation, the full set of Product Qualification functions would be invoked against each instance of the product in order to determine the inferred product type. Given the product model composability, a single product instance may be qualified as more than one type: for example in an Interest Rate Swaption, both the Option and the underlying Interest Rate Swap would be qualified.
- Please browse the Product Qualification functions in the model (look in the namespace `cdm.product.qualification:func`) and, in particular, try to understand those for the products with which you are most familiar.

D

The fourth module in this series explains how CDM is used within financial services to deliver the stated benefits.

1

Overview of the CDM and the CDM components

Complete

2

Data Interfaces to the CDM and JSON

Complete

3

Documentation and The Object Builder

Complete

4

Introduction to Functions

Complete

5

The CDM for Technologists

Start

FINOS CDM Distribution (Open Source)

github.com/finos/common-domain-model

Model Definition

Data Model

- Product Model
- Event Model
- Legal Agreement Model

Process Model

- Calculation
- Event Logic
- Event / Product Qualification

Reporting

- Eligibility Rule
- Reportable Field
- Regulatory Reference

Data Validation

- Condition
- Special Syntax: eg choice, one-of

Annotation

- Meta-Data: eg 'key' / 'reference'
- Function Type

Namespace

- Model Partition
- Description
- Hierarchy

Interfaces

Mapping / Synonym

- FpML
- DTCC
- CME
- ISO 20022

Examples (.json)

- Product samples
- Event samples

Executable Code Distribution

Organised into namespaces

Data Model

- Product Model
- Event Model
- Legal Agreement

Process Model

- Calculation
- Event Logic
- Event / Product Qualification

Data Validation

- Condition
- Syntax: e.g. 'choice', 'one-of'

Reporting

- Eligibility Rule
- Reportable Field

Default Implementation

Hand-crafted code (not auto-generated)

Meta-Data

- Processor
- 'Key' hash code implementation and override
- Meta-Type
- Event Effect Post-Processor

Function

- Calculation period
- Days in period
- Resolve index

Serialiser

- JSON

Documentation cdm.finos.org

CDM Homepage

- Apache license via GitHub
- Various model definitions
- Documentation

CDM Object Builder

- Create, import, share or inspect CDM JSON
- cdm-object-builder.finos.org

Integration Test Pack

Ingestion Test (.xml, .json)

- Input samples: FpML / DTCC / CME
- Output + assertion files (JSON)

Function Test

- Function input samples (XML)
- Output + assertion files (JSON)

- We would like to provide you with a quick overview of some of the technical capabilities provided in the CDM distribution, and available to everyone at FINOS.
- These capabilities, while most likely to be of use to technology teams, deliver the power of the CDM and, through this overview, we hope to demonstrate the breadth and depth of capabilities available to help member firms leverage the CDM to deliver efficiency in their organisations.

Interfaces

Mapping / Synonym

- FpML
- DTCC
- CME
- ISO 20022

Examples (.json)

- Product samples
- Event samples

- In order to facilitate the translation of existing industry messages (based on open standards or proprietary ones) into the CDM, the CDM is mapped to a set of those alternative data representations using the Rosetta DSL synonym feature.
- Synonyms added throughout the model are combined to map an input document into the output Rosetta document. The synonyms can be used to generate an Ingestion Environment, a Java library which, given an input document, will output the resulting Rosetta document.
- Snippet example from the mapping of a confirmation expressed in FpML:

```
synonym source FpML_5_Confirmation_To_TradeState extends FpML
{
  PerformancePayout:
    [meta "id"]
    + payerReceiver
      [hint "fixedLeg"]
      [hint "floatingLeg"]
    - priceQuantity
    + priceQuantity
      [value "vegaNotional" set when path = "fxVolatilitySwap"]
      [value "rateOfReturn"]
      [value "underlyer" set when path = "dividendLeg"]
      [value "notional"] // when monetary amount
      [hint "notional"] // for FX Variance Swap
    + paymentDates
      [value "paymentDates" path "rateOfReturn"]
    + underlier
      [value "singleUnderlyer" path "underlyer" , "underlyer"]
    + fxFeature
      [value "fxFeature"]
}
```

Want to know more?

See the [list of sources](#) currently available in the CDM distribution.

Read about [mapping and synonyms](#) in the Rosetta DSL documentation.

Executable Code Distribution*Organised into namespaces***Data Model**

- Product Model
- Event Model
- Legal Agreement

Process Model

- Calculation
- Event Logic
- Event / Product Qualification

Data Validation

- Condition
- Syntax: e.g. 'choice', 'one-of'

Reporting

- Eligibility Rule
- Reportable Field

- The CDM Process Model has been designed to translate the technical standards that support those industry processes into a standardised machine-readable and machine-executable format.
- Machine readability and executability is crucial to eliminate implementation discrepancy between market participants and increase interoperability between technology solutions. It greatly minimises the cost of adoption and provides a blueprint on which industry utilities can be built..
- The data and process model definitions of the CDM are systematically translated into executable code using purpose-built code generation technology. The CDM executable code is available in a number of modern, widely adopted and freely available programming languages and is systematically distributed as part of the CDM release.

Want to know more?

See the [list of code generation languages](#) currently available.

The [code generation](#) process is based on the Rosetta DSL and is described in the documentation.

Default Implementation*Hand-crafted code (not auto-generated)***Meta-Data Processor**

- 'Key' hash code implementation and override
- Meta-Type
- Event Effect Post-Processor

Function

- Calculation period
- Days in period
- Resolve index

Serialiser

- JSON

- In addition to the automatically generated code that represents the model itself, the CDM distribution also includes supporting code that provides
 - Meta data processing – that is a method to link external identifiers in data to representations in the CDM
 - Functions – code to support the base functions in the model
 - Serialisation – that is the production of JSON from the model.

Developer guides

Developers considering the CDM should look at the [CDM Java Distribution Guidelines](#) to understand how to [download](#) and use the Java version of the CDM.

D	1	Overview of the CDM and the CDM components	Review
	2	Data in the CDM and JSON	Review
	3	Data Interfaces to the CDM and JSON	Review
	4	Documentation and The Object Builder	Review
	5	The CDM for Technologists	Review
	6	Take your knowledge check	Start

Q: The FINOS CDM Distribution is Open Source and consists of the following components:

- A1 Model Definition: Data, Process, Events
- A2 Integration Test Pack: Ingestion tests, Function Tests
- A3 Executable Code: Model, Functions, Serialisation
- A4 Interfaces: mappings and synonyms
- A5 All of the above

Q: The FINOS CDM Distribution is Open Source and consists of the following components:

- A1 Model Definition: Data, Process, Events
- A2 Integration Test Pack: Ingestion tests, Function Tests
- A3 Executable Code: Model, Functions, Serialisation
- A4 Interfaces: mappings and synonyms
- A5 All of the above

The FINOS CDM Distribution is Open Source and consists of all of the components listed, and more.

Q: Which of the following statements is untrue about JSON?

- A1 JSON is easy for humans to read and easy for machines to parse and generate
- A2 JSON is short for JavaScript Object Notation
- A3 JSON is supported by all modern programming languages
- A4 JSON is hard for humans to read but easy for machines to execute
- A5 JSON is a lightweight data interchange format

Q: Which of the following statements is untrue about JSON?

- A1 JSON is easy for humans to read and easy for machines to parse and generate
- A2 JSON is short for JavaScript Object Notation
- A3 JSON is supported by all modern programming languages
- A4 JSON is hard for humans to read but easy for machines to execute
- A5 JSON is a lightweight data interchange format

JSON (JavaScript Object Notation) is a standard, lightweight, text-based, human-readable data interchange format used to exchange data between applications and servers.

It is easy for humans to read.

It is also easy for machines to parse and generate.

Q: The Rosetta Platform consists of several applications; which of the following is not a Rosetta application?

- A1 Rosetta Workspace – a user-specific modelling area
- A2 Rosetta Design – a model editor
- A3 Rosetta Translate – mapping between existing systems, databases, message formats, and the CDM
- A4 Rosetta Engine – run functional logic on physical data
- A5 Rosetta Object Builder – view, create and manage JSON data

Q: The Rosetta Platform consists of several applications; which of the following is not a Rosetta application?

- A1 Rosetta Workspace – a user-specific modelling area
- A2 Rosetta Design – a model editor
- A3 Rosetta Translate – mapping between existing systems, databases, message formats, and the CDM
- A4 Rosetta Engine – run functional logic on physical data
- A5 Rosetta Object Builder – view, create and manage JSON data

The first four answers are all applications that are delivered on the [Rosetta Platform](#).

The Object Builder is a component of the CDM that is part of the FINOS Distribution, either as code downloadable from [GitHub](#) or as a service available at:

<https://cdm-object-builder.finos.org>.

Q: Which of the following is NOT provided in the CDM Model Definition?

A1 CDM Event Model

A2 CDM Product Model

A3 CDM Issuer Model

A4 CDM Agreement Model

A5 CDM Event & Product Qualification

Q: Which of the following is NOT provided in the CDM Model Definition?

A1 CDM Event Model

A2 CDM Product Model

A3 CDM Issuer Model

A4 CDM Agreement Model

A5 CDM Event & Product Qualification

“CDM Issuer Model”. Is not part of the CDM Model Definition but all the other items listed are provided.

Issuers can be modelled in the CDM, depending on the product and events they are interacting with.

Q: In CDM 5, there is a *function* named **InterestRatePayoutCurrency**. What does this function return?

- A1 A list of interest rate payouts
- A2 A list of currencies collected from the payouts
- A3 An **InterestRatePayoutCurrency**
- A4 An **InterestRatePayout**
- A5 A string representing a single currency

Q: In CDM 5, there is a *function* named **InterestRatePayoutCurrency**. What does this function return?

- A1 A list of interest rate payouts
- A2 A list of currencies collected from the payouts
- A3 An **InterestRatePayoutCurrency**
- A4 An **InterestRatePayout**
- A5 A string representing a single currency

This function can be found in the model by searching for it in GitHub or Rosetta.

The function definition shows that the **output** is a **currency string** with multiple cardinality – i.e., it will return a list of currencies, retrieved from the **payout** inputs.

```
func InterestRatePayoutCurrency:  
  inputs:  
    interestRatePayouts InterestRatePayout (0..*)  
    <"List of interest rate payouts.">  
  output:  
    currency string (0..*)  
    <"List of currencies collected from the payouts.">
```


Q: Which of the following statements is inaccurate?

- A1 The CDM does not support Python
- A2 The CDM provides Java code for serialisation to JSON
- A3 No code must be written to adopt the CDM
- A4 FpML data can be imported into CDM
- A5 The CDM provides integrated test packs

Q: Which of the following statements is inaccurate?

- A1 The CDM does not support Python
- A2 The CDM provides Java code for serialisation to JSON
- A3 No code must be written to adopt the CDM
- A4 FpML data can be imported into the CDM
- A5 The CDM provides integrated test packs

Python is one of the many languages supported in the CDM Distribution with *code generators*, so this statement is not accurate.

All of the other statements are accurate and are examples of the technical capabilities that are provided in the CDM Distribution.

Q: One of the critical calculations performed on trades in finance is day count; which of the following statements about the CDM is true?

- A1 The CDM enables Java code to be used for day count calculations
- A2 The CDM only supports ISDA's day count definitions
- A3 Day count calculations must be implemented alongside the CDM using Java or any other modern programming language
- A4 The CDM contains functions and Java code which standardise the calculation of day counts in the model
- A5 Day count calculations are implemented in the CDM using JSON

Q: One of the critical calculations performed on trades in finance is day count; which of the following statements about the CDM is true?

- A1 The CDM enables Java code to be used for day count calculations
- A2 The CDM only supports ISDA's day count definitions
- A3 Day count calculations can be implemented alongside the CDM using Java or any other modern programming language
- A4 The CDM contains functions and Java code which standardise the calculation of day counts in the model
- A5 Day count calculations are implemented in the CDM using JSON

All of the common day count definitions have been implemented as functions in the CDM, backed by Java code which is part of the CDM Distribution.

This means they can be directly invoked in the model, accessed via an API, and are provided as ready-to-use code in Java and several other languages.

Q: Which of the following statements about the CDM Object Builder is not true?

- A1 It enables the construction of test data for the CDM
- A2 It can be integrated into in-house or commercial applications
- A3 It enables end users to visualise data stored in the CDM
- A4 It is available on payment of a licence fee or service contract
- A5 It can be used to validate the correctness of JSON data

Q: Which of the following statements about the CDM Object Builder is not true?

- A1 It enables the construction of test data for the CDM
- A2 It can be integrated into in-house or commercial applications
- A3 It enables end users to visualise data stored in the CDM
- A4 It is available on payment of a licence fee or service contract
- A5 It can be used to validate the correctness of JSON data

The CDM Object Builder is open source and available to all participants without payment and without a contract.

All of the other statements are true and are examples of use cases for the CDM Object Builder.

How the CDM Works

D	1	Overview of the CDM and the CDM components	Review
	2	Data in the CDM and JSON	Review
	3	Data Interfaces to the CDM and JSON	Review
	4	Documentation and The Object Builder	Review
	5	The CDM for Technologists	Review

Well Done! You have completed the fourth module

You can review any of the complete sections again or move on to the next module.

Done